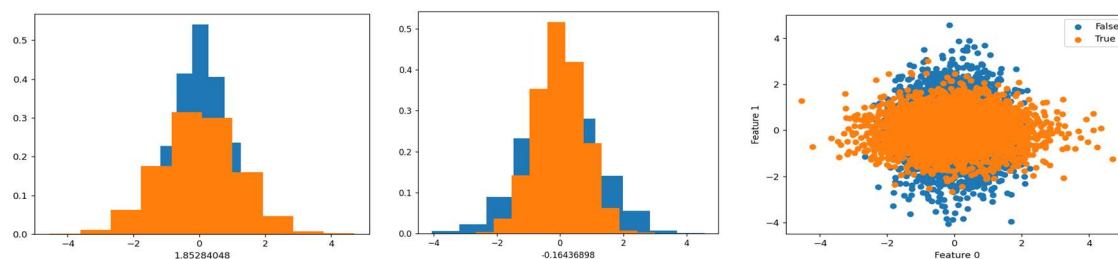


DOMANDE PROGETTO ML

Dataset:

The project focuses on a binary classification task for fingerprint spoofing detection, with the objective of distinguishing genuine fingerprint images from counterfeit ones. The dataset consists of labeled samples belonging to two classes: genuine fingerprints (True, label 1) and fake fingerprints (False, label 0). Each sample is represented by a 6-dimensional feature vector extracted from the original fingerprint image through a feature extraction process that captures high-level characteristics. Each row corresponds to a sample, where the first six values represent the features and the last value represents the class label (0 or 1).

1. Analysis of the first two features



The histograms of the first two features show a strong overlap between the two classes, especially in the central region of the distributions (approximately between -1 and 1). The scatter plots also confirm that the samples from the two classes occupy almost the same region of the feature space, making these features poorly discriminative for classification.

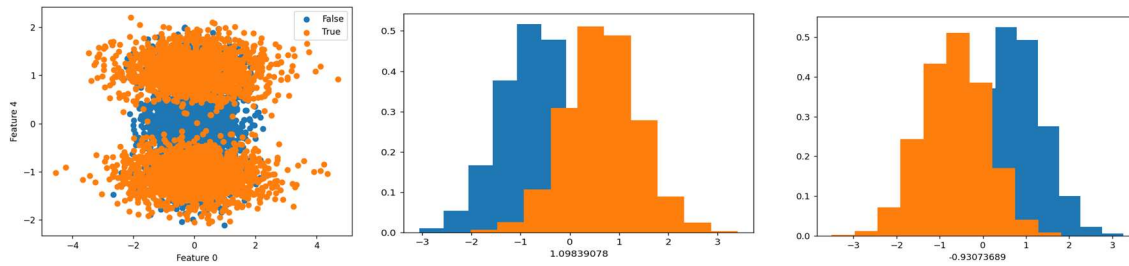
The means of the two classes are very similar, since both distributions are centered around values close to zero. The variances are also comparable, although one class appears slightly more dispersed than the other.

The histograms show a single dominant peak for both classes, indicating unimodal distributions.

Overall, the first two features do not provide good class separation because they exhibit:

- strong overlap,
- similar means,
- similar variances,
- unimodal distributions: the samples of a class are generated from a single main group. This implies that both classes have very similar distributions for the first features and these features alone are not very useful for classification.

2. Analysis of the third and fourth features



The third and fourth features show a clearer separation between the classes compared to the first two features. The overlap is reduced, although some samples from the two classes still intersect in the central region of the distributions. The means of the classes are noticeably different, since the peaks of the histograms are shifted with respect to each other. The variances are similar, with both classes showing comparable dispersion.

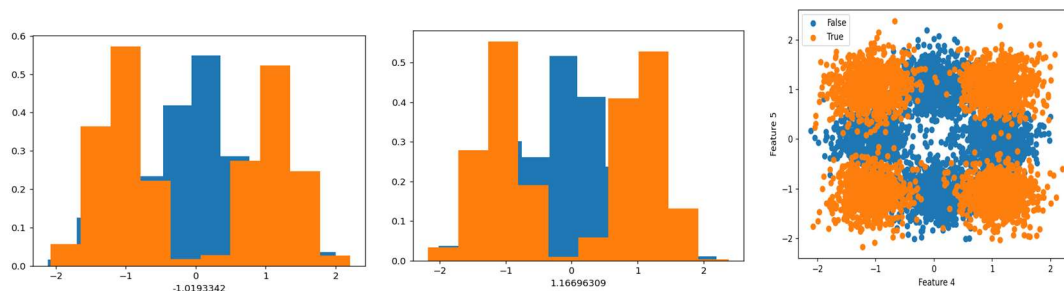
The histograms remain unimodal, as each class still presents a single dominant peak.

The scatter plots confirm that these features are more informative and discriminative than the first two ones, although they are still insufficient for a complete separation of the classes.

In summary, features 3 and 4:

- show lower overlap,
- have more distinct means,
- maintain similar variances,
- remain unimodal.

3. Analysis of the last two features



The last two features reveal a more complex structure of the dataset. The classes still overlap in the central region, but several areas exist where one class dominates the other.

A key observation from the histograms is that one class exhibits a bimodal distribution, with two distinct peaks, while the other class remains approximately unimodal. This suggests the presence of multiple subgroups within one class.

The scatter plots provide even clearer evidence of this structure. In particular:

- one class forms multiple distinct clusters,

- the other class is mainly concentrated around the center.

In the Feature 4–5 scatter plot, four distinct clusters are clearly visible for the True class. The False class instead appears concentrated near the center of the feature space.

The overlap between classes is mainly limited to the central region, whereas the external regions show a much better separation. The multimodal structure observed in the histograms is consistent with the cluster structure visible in the scatter plots: the multiple lateral clusters generate multiple peaks when each feature is analyzed individually.

Overall, the last two features provide the most informative representation of the dataset because they highlight:

- the presence of multiple clusters,
- bimodal behavior in one class,
- a more structured distribution of the samples,
- improved class separability compared to the previous features.

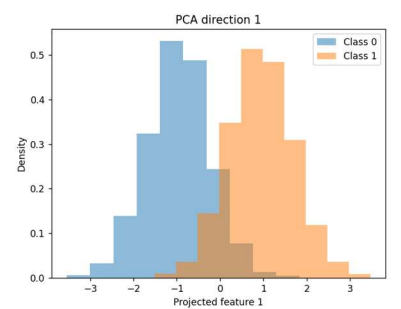
PCA AND LDA

PCA

1) What do you observe from the PCA histograms and scatter plots?

After applying PCA to the 6-dimensional fingerprint spoofing dataset, the projected features show different distributions along the principal directions.

Observations from the histograms

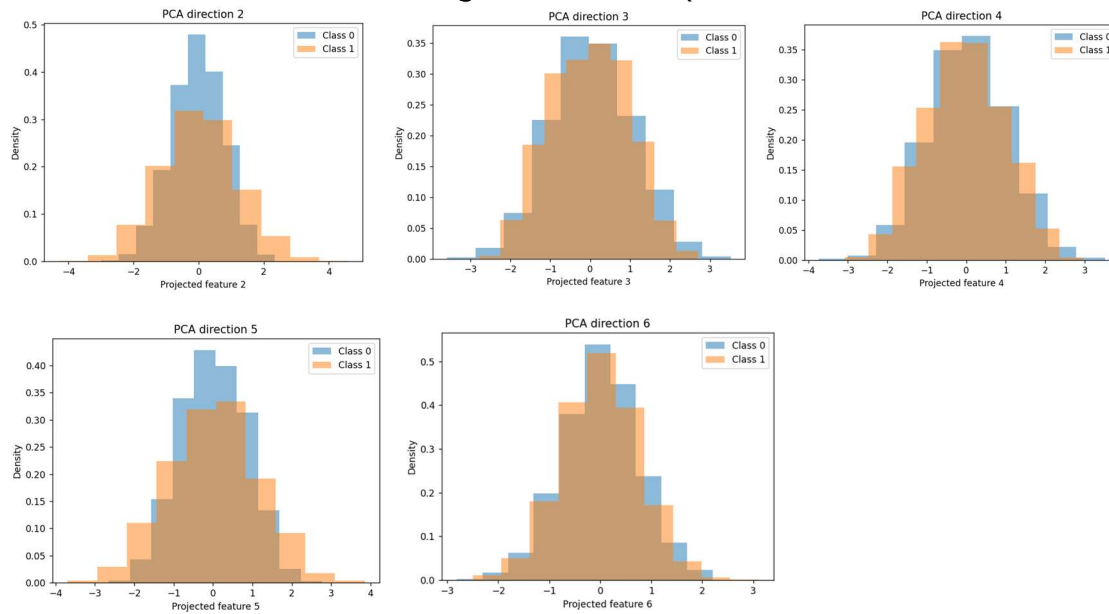


PCA direction 1 (largest variance) provides the clearest visible separation between the two classes:

Class 0 (fake fingerprints) is mostly concentrated on negative projected values.

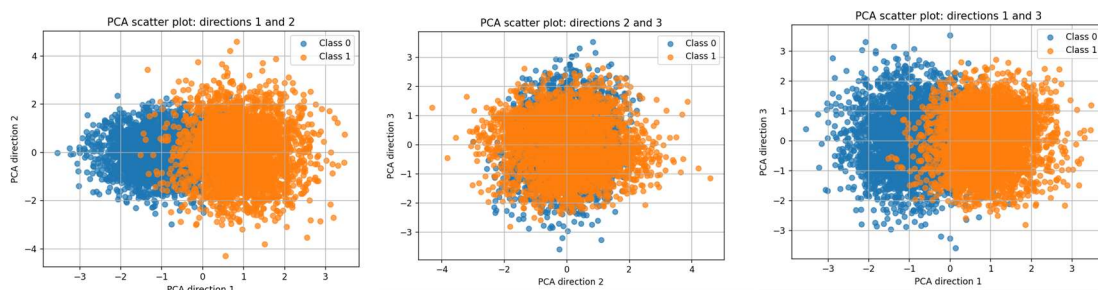
Class 1 (genuine fingerprints) is mostly concentrated on positive projected values.

However, there is still a significant overlap between the two distributions.



PCA directions 2–6 contain progressively less discriminative information: the histograms become increasingly similar, the overlap between the classes increases, the classes appear highly mixed. This means that the first principal component captures most of the variance related to the global structure of the dataset, while the remaining components mainly describe variability inside each class.

Observations from the scatter plots



The scatter plots confirm the histogram analysis: The projection using PCA directions 1 and 2 shows the most visible separation between genuine and fake fingerprints. Scatter plots involving higher-order PCA directions show much stronger overlap. The first PCA direction contains most of the useful visible information for discrimination. However, even in the best projections, the separation between classes is only partial.

2) What are the effects of PCA on the class distribution?

PCA changes the representation of the data by projecting the samples onto a new orthogonal coordinate system. PCA maximizes the variance of the projected data: $\text{Var}(w^T x)$. As a consequence: the shapes of the feature distributions change, the projected histograms look different, some PCA directions become more informative than others, variance is redistributed across the new coordinates. The first principal

components contain most of the global variance, while the later components contain less relevant information. However, PCA is an unsupervised technique: it does not use class labels, it does not explicitly maximize class separability. Therefore, PCA mainly reorganizes the variance of the dataset rather than improving classification directly. Variance is redistributed across the new coordinates, some directions contain more global variance, other directions contain very little useful information for classification.

3) Can you spot different clusters inside each class?

No clear internal subclusters are visible inside the classes. The scatter plots show relatively compact and continuous distributions rather than multiple separated groups. Some local density variations can be observed, but they do not correspond to clearly separated clusters. Therefore, PCA does not reveal strong hidden substructures inside the classes.

4) Have the clusters really changed recalling PCA with all 6 dimensions?

No. PCA was applied while keeping all 6 dimensions, therefore the transformation corresponds only to a rotation of the original feature space. This means that: the intrinsic geometry of the dataset is preserved, distances between samples remain unchanged, relationships among samples are not fundamentally modified. The apparent changes observed in histograms and scatter plots are caused only by the change of coordinates introduced by PCA. Therefore, the clusters themselves have not actually changed.

5) Have the classes become more separated?

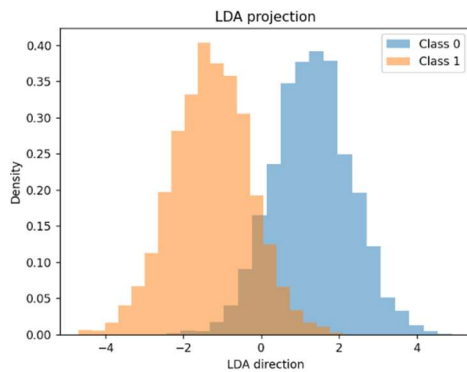
No. Although the first PCA direction visually shows better separation between genuine and fake fingerprints, PCA does not optimize class discrimination. PCA only maximizes total variance, not class separability. As a result: some projections may appear more discriminative, but the global structure of the dataset remains unchanged, the overlap between the classes still exists. The apparent improvement in separation is mainly a visualization effect due to the new coordinate system.

LDA analysis

Since the dataset contains only two classes (genuine fingerprints and fake fingerprints), LDA produces a **single projection direction**. Unlike PCA, which maximizes the overall variance of the data, LDA is a **supervised** technique that uses the class labels to find the direction that best separates the classes. LDA finds the direction that maximizes the ratio between:

- the distance between class means,
- and the variance within each class

1) What do you observe from the LDA histogram?



From the histogram, we observe that the two classes are projected into two mostly distinct regions:

- **Class 0** is mainly concentrated on **positive values**.
- **Class 1** is mainly concentrated on **negative values**.
- The two distributions are more separated than in PCA.
- The overlap is relatively small and mainly concentrated around the central region near zero.

This shows that the LDA direction is highly discriminative.

2) Do the classes overlap over the found direction?

Yes, but only partially. The classes are not perfectly separated, because there is still some overlap around the center of the histogram. However, compared to PCA: the overlap is smaller, the class means are farther apart, the distributions are more compact.

3) Compared to the histograms of the 6 original features, is LDA finding a good direction with little class overlap?

Yes. Compared to the original feature histograms (PCA), the LDA projection gives a clearer separation between the two classes. This happens because LDA combines the original 6 features into one discriminative direction. Instead of looking at each feature separately, it finds the direction that maximizes the ratio:

between-class variance / within-class variance

Therefore:

- samples from different classes are projected farther apart,
- samples from the same class are kept relatively compact,
- the class overlap is reduced.

So LDA finds a good direction with little class overlap.

4) Is LDA increasing the separation between classes?

LDA is not increasing the separation between classes, it's finding a direction in the original feature space along which the classes are already well separated. In other words, LDA reveals the most discriminative direction already present in the data.

5) Why is LDA better than PCA here?

PCA and LDA have different goals. **PCA** finds directions with high variance. Some of this variance may correspond to class separation, but this is accidental because PCA does not use labels. **LDA** uses the class labels and searches specifically for a direction that minimizes class overlap. This is why LDA is more targeted for classification. In this dataset, PCA direction 1 already showed some separation, but LDA gives a cleaner and more discriminative projection because the dataset was already partially separable in the original feature space.

LDA as a Classifier

The dataset was divided into:

- a **training set**, used to compute the LDA projection direction,
- and a **validation set**, used to evaluate the classifier performance.

After computing the LDA direction, the orientation was selected such that:

- the projected mean of **class 1 (True / genuine fingerprints)** is larger than the projected mean of **class 0 (False / fake fingerprints)**.

The classification threshold was chosen as the midpoint between the projected class means:

$$t = \frac{\mu_0 + \mu_1}{2}$$

The obtained threshold was:

$$t = -0.0185$$

The classifier decision rule is therefore:

- if the LDA projection $\geq -0.0185 \rightarrow$ predict class 1
- otherwise $\rightarrow$ predict class 0

Classification Results

Using this default threshold, the classifier correctly classified about:

90.70% of the validation samples.

The obtained error rate was: 9.30% corresponding to:

- **186 classification errors out of 2000 samples.**

This shows that the LDA projection provides a good discriminative representation for the fingerprint spoofing problem.

What happens when changing the threshold?

After modifying the classification threshold, only a very small improvement in performance was observed.

The error rate changed from:

- 9.30% → 9.10%

This means that the original threshold was already very close to the optimal decision boundary.

Why is the midpoint threshold already good?

The threshold chosen as the average of the projected class means works well because:

- the projected distributions are relatively symmetric,
- the classes have similar spread,
- the overlap region is approximately centered near the midpoint.

Therefore, the midpoint between the class means already provides a reasonable balance between the two classes.

What do we conclude from threshold tuning?

Changing the threshold can slightly rebalance the classification errors:

- one class may become classified slightly better,
- while the other may become slightly worse.

However, threshold tuning cannot completely remove the classification errors because the two projected distributions still overlap. The remaining errors are mainly caused by the intrinsic overlap between genuine and fake fingerprint samples rather than by a poor threshold choice. Therefore:

- LDA successfully finds a highly discriminative direction,
- the midpoint threshold is already close to optimal,
- and threshold tuning only provides marginal improvements in accuracy.

PCA + LDA Classification

To evaluate whether PCA preprocessing can improve classification performance, PCA was first applied to the training data and then LDA was used for classification.

For each number of retained PCA dimensions m :

1. PCA was estimated using only the training data.
2. Both training and validation samples were projected onto the PCA subspace.
3. LDA was applied to the PCA-projected training data.
4. Validation samples were classified using the LDA classifier.
5. The validation error rate was computed.

The number of retained PCA dimensions was varied from $m = 1$ to $m = 6$.

Validation Error Rates

PCA Dimensions (m)	Error Rate
1	9.35%
2	9.25%
3	9.25%
4	9.25%
5	9.30%
6	9.30%

What do you observe?

The results show that PCA preprocessing produces only very small changes in classification performance. The best performance was obtained for: $m = 2,3,4$ with an error rate of: 9.25%. This is only slightly better than the original LDA classifier without PCA. When all 6 PCA dimensions are retained, the performance becomes identical to standard LDA without PCA: 9.30%. This confirms that PCA with full dimensionality is simply a rotation of the original feature space and does not change the intrinsic discriminative structure of the dataset.

Can you find values of m that improve the validation accuracy?

Yes, but only marginally.

Using: $m = 2,3,4$ slightly improves the validation accuracy compared to LDA alone. However, the improvement is extremely small:

- from 9.30%
- to 9.25%

Therefore, PCA preprocessing does not significantly improve classification performance.

Why does aggressive dimensionality reduction worsen performance?

Using only one PCA direction ($m = 1$) gives the highest error rate: 9.35%. This happens because PCA selects directions with maximum variance, but low-variance directions may still contain useful discriminative information for classification. By keeping only one component, some relevant class information is lost.

Is PCA beneficial when combined with LDA?

Overall, PCA is not significantly beneficial for this task. The dataset already contains a strong discriminative direction, and LDA alone is already able to identify it effectively. PCA preprocessing only produces very small performance variations and does not fundamentally improve the classification accuracy. In particular:

- PCA does not create new discriminative information,
- PCA mainly reorganizes the variance of the dataset,
- LDA already finds the most discriminative direction in the original feature space.

Project - question example 4

Given the following functions (assume these functions are already implemented unless specified):

- `trainPCA`: trains a PCA model
- `applyPCA`: applies a PCA model to some data
- `trainClassifier(D, L)`: trains a given classifier from the data matrix `D` and the label vector `L`; returns an object containing the trained model parameters
- `scoreClassifier(clsModel, D)`: computes the array of scores for classifier `clsModel` (as returned by the function `trainClassifier`) for the samples in data matrix `D`
- `evaluateScores(S, L)`: computes a performance metric (e.g. minimum DCF) over the score array `S` with label vector `L`

a) Provide a possible signature and an implementation of the functions `trainPCA` and `applyPCA`, briefly explaining also the function parameters and the return value.

b) Using these functions, write the Python code to:

- Train the classifier on a training set, optimizing the PCA dimension with respect to the provided metric function using a single-fold cross-validation approach
- Evaluate its performance on an evaluation set.

Assume that you have at your disposal a training set, already divided in model training data (`DTR`, `LTR`) and validation data (`DVAL`, `LVAL`), and an evaluation set (`DTE`, `LTE`). `DTR`, `DVAL` and `DTE` are data matrices, with samples organized as column vectors, whereas `LTR`, `LVAL` and `LTE` are arrays containing the corresponding labels. To select the PCA dimension m consider all possible values of m that are compatible with the dimension of the feature vectors. Assume that the classifier is invariant to affine transformations, that it does not include hyper-parameters to tune, and that PCA is the only kind of pre-processing to analyze.

Summary of main numpy (np) and scipy employed in the laboratories

<code>s, U = np.linalg.eigh(C)</code> returns the array of eigenvalues <code>s</code> in ascending order and the matrix of corresponding eigenvectors <code>U</code> of a real symmetric matrix <code>C</code>	<code>s, v = np.linalg.slogdet(C)</code> returns the sign <code>s</code> and the logarithm of the absolute value <code>v</code> of the determinant of matrix <code>C</code>
<code>U, s, Vh = np.linalg.svd(C)</code> returns the array of singular values <code>s</code> in descending order, the corresponding matrix of left singular vectors <code>U</code> and the corresponding transposed matrix of right singular vectors <code>Vh</code>	<code>v = scipy.special.logsumexp(M, axis=k)</code> computes in a numerically more stable way <code>np.log(np.sum(np.exp(a), axis=k))</code>
	<code>v = np.logaddexp(a, b)</code> computes in a numerically more stable way <code>np.log(np.exp(a) + np.exp(b))</code>

□) We have to TRAIN PCA on DTR:

```
def vcol(x):
```

```

return x.reshape((x.size, 1))

def vrow(x):
    return x.reshape((1, x.size))

```

```

def trainPCA(D, m):
    mu = vcol(D.mean(axis=1))
    DC = D - mu
    C = (DC @ DC.T) / D.shape[1]

```

```

    U, s, Vh = np.linalg.svd(C)
    P = U[:, 0:m]

```

```

    return mu, P

```

- D is a matrix whose columns are samples
- m is the number of PCA dimensions to keep
- P is the projection matrix

□) Apply PCA on DTR and DVAL

```

def applyPCA(D, mu, P):
    return P.T @ (D - mu)

```

It simply project the samples on the PCA directions.

-) Train classifier on projected DTR
-) Evaluate on projected DVAL
-) Choose the best m
-) Apply the final model to DTE
-) Compute the final evaluation performance

The best m is selected using the validation set. We talk about single-fold-cross-validation approach because the model is trained on DTR and validate on DVAL.

```

def train_PCA_classifier_with_validation(DTR, LTR, DVAL, LVAL):
    best_m = None
    best_metric = None
    best_model = None

```

```

    for m in range(1, DTR.shape[0] + 1):

```

```

        # Train PCA only on training data
        pcaModel = trainPCA(DTR, m)

```

```

        # Apply PCA to training and validation data
        DTR_pca = applyPCA(pcaModel, DTR)
        DVAL_pca = applyPCA(pcaModel, DVAL)

```

```
# Train classifier on PCA-projected training data
clsModel = trainClassifier(DTR_pca, LTR)
```

```
# Compute validation scores
SVAL = scoreClassifier(clsModel, DVAL_pca)
```

```
# Evaluate validation performance
metric = evaluateScores(SVAL, LVAL)
```

```
if best_metric is None or metric < best_metric:
    best_metric = metric
    best_m = m
    best_model = clsModel
```

```
return best_m, best_metric
```

FINAL EVALUATION:

```
best_m, best_metric = train_PCA_classifier_with_validation(
    DTR, LTR,
    DVAL, LVAL
)
```

```
# Train PCA again using the selected dimension
pcaModel = trainPCA(DTR, best_m)
```

```
# Project training and evaluation data
DTR_pca = applyPCA(pcaModel, DTR)
DTE_pca = applyPCA(pcaModel, DTE)
```

```
# Train final classifier
clsModel = trainClassifier(DTR_pca, LTR)
```

```
# Score evaluation data
STE = scoreClassifier(clsModel, DTE_pca)
```

```
# Evaluate final performance
eval_metric = evaluateScores(STE, LTE)
```

If I have to apply LDA binary classifier with PCA selection:

```
def trainLDA(D, L):
    Sb, Sw = compute_Sb_Sw(D, L)
    s, U = scipy.linalg.eigh(Sb, Sw)
    W = U[:, ::-1][:, 0:1]
```

```
DLDA = W.T @ D
```

```
if DLDA[0, L == 1].mean() < DLDA[0, L == 0].mean():
    W = -W
```

```
return W
```

Since the problem is binary, LDA returns one projection direction W . The orientation is selected so that class 1 has larger projected mean than class 0.

```
def compute_Sb_Sw(D, L):
    muGlobal = vcol(D.mean(axis=1))
    Sb = 0
    Sw = 0
    for c in np.unique(L):
        Dc = D[:, L == c]
        muClass = vcol(Dc.mean(axis=1))
        # Between-class scatter
        diff = muClass - muGlobal
        Sb += Dc.shape[1] * (diff @ diff.T)
        # Within-class scatter
        Dc_centered = Dc - muClass
        Sw += Dc_centered @ Dc_centered.T
    Sb /= D.shape[1]
    Sw /= D.shape[1]
    return Sb, Sw
```

```
def applyLDA(D, W):
    return W.T @ D
```

It projects the data onto the LDA direction and returns LDA scores.

```
best_m = None
best_metric = None

for m in range(1, DTR.shape[0] + 1):

    # Train PCA only on model training data
    mu_pca, P = trainPCA(DTR, m)

    # Apply the same PCA model to training and validation data
    DTR_pca = applyPCA(DTR, mu_pca, P)
```

```

DVAL_pca = applyPCA(DVAL, mu_pca, P)

# Train LDA on PCA-projected training data
W = trainLDA(DTR_pca, LTR)

# Compute validation scores
SVAL = applyLDA(DVAL_pca, W).ravel()

# Evaluate validation performance
metric = evaluateScores(SVAL, LVAL)

if best_metric is None or metric < best_metric:
    best_metric = metric
    best_m = m

```

In this case we select the PCA dimensionality m using the validation set. PCA is trained only on DTR, because the validation data must not influence the training process.

After selecting the best PCA dimension, we evaluate the final system on the evaluation set:

```

# Train PCA again with the selected dimension
mu_pca, P = trainPCA(DTR, best_m)

# Apply PCA to training and evaluation data
DTR_pca = applyPCA(DTR, mu_pca, P)
DTE_pca = applyPCA(DTE, mu_pca, P)

# Train final LDA classifier
W = trainLDA(DTR_pca, LTR)

# Compute evaluation scores
STE = applyLDA(DTE_pca, W).ravel()

# Evaluate final performance
eval_metric = evaluateScores(STE, LTE)

```

If prof asks for **predicted labels**, than we need a **threshold**.

After training LDA, the validation samples are projected onto the LDA direction. Since the problem is binary, LDA produces one-dimensional scores. First, we choose the orientation of the LDA direction so that class 1 has larger projected mean than class 0:

```

W = trainLDA(DTR_pca, LTR)

DTR_Ida = applyLDA(DTR_pca, W)

if DTR_Ida[0, LTR == 1].mean() < DTR_Ida[0, LTR == 0].mean():
    W = -W
    DTR_Ida = applyLDA(DTR_pca, W)

```

Then we project the validation data:

```

DVAL_Ida = applyLDA(DVAL_pca, W)

```

The default threshold is chosen as the average between the projected class means on the training set:

```
threshold = (
    DTR_Ida[0, LTR == 0].mean()
    + DTR_Ida[0, LTR == 1].mean()
) / 2.0
```

Then predictions are computed as:

```
PVAL = np.zeros(LVAL.shape, dtype=np.int32)
```

```
PVAL[DVAL_Ida[0] >= threshold] = 1
```

```
PVAL[DVAL_Ida[0] < threshold] = 0
```

Finally, the error rate is:

```
errors = (PVAL != LVAL).sum()
```

```
error_rate = errors / LVAL.size * 100
```

Threshold tuning

To improve the threshold, we can test many possible values on the validation set and choose the one that gives the lowest error rate.

This follows your function:

```
def tune_threshold(DVAL_Ida, LVAL):

    thresholds = np.linspace(DVAL_Ida.min(), DVAL_Ida.max(), 1000)

    best_threshold = None
    best_errors = LVAL.size
    best_error_rate = None

    for t in thresholds:

        PVAL = np.zeros(LVAL.shape, dtype=np.int32)

        PVAL[DVAL_Ida[0] >= t] = 1
        PVAL[DVAL_Ida[0] < t] = 0

        errors = (PVAL != LVAL).sum()
        error_rate = errors / LVAL.size * 100

        if errors < best_errors:
            best_errors = errors
            best_error_rate = error_rate
            best_threshold = t

    return best_threshold, best_errors, best_error_rate
```

To apply this:

```
# Tune threshold on validation data
```

```
threshold, errors, error_rate = tune_threshold(DVAL_Ida, LVAL)
```

```

if best_error_rate is None or error_rate < best_error_rate:
    best_error_rate = error_rate
    best_m = m
    best_threshold = threshold

```

Then EVALUATE the final model on the EVALUATION SET:

```

mu_pca, P = trainPCA(DTR, best_m)

DTR_pca = applyPCA(DTR, mu_pca, P)
DTE_pca = applyPCA(DTE, mu_pca, P)

W = trainLDA(DTR_pca, LTR)

DTR_Ida = applyLDA(DTR_pca, W)

if DTR_Ida[0, LTR == 1].mean() < DTR_Ida[0, LTR == 0].mean():
    W = -W
DTR_Ida = applyLDA(DTR_pca, W)

DTE_Ida = applyLDA(DTE_pca, W)

PTE = np.zeros(LTE.shape, dtype=np.int32)

PTE[DTE_Ida[0] >= best_threshold] = 1
PTE[DTE_Ida[0] < best_threshold] = 0

errors = (PTE != LTE).sum()
error_rate = errors / LTE.size * 100

```

Gaussian

```

def compute_mu_var_1D(x):
    """
    x is a 1D array containing all samples of one feature
    for one specific class.
    """
    mu = x.mean()
    var = ((x - mu) ** 2).mean()
    return mu, var

# Compute the log-density (numerically much more stable).

def logpdf_GAU_1D(x, mu, var):
    """
    Computes the log-density of a 1D Gaussian.
    """
    return -0.5*np.log(2*np.pi) - 0.5*np.log(var) - ((x - mu) ** 2) / (2 *
var)

```

Step 1 — Goal

For each **class** and each **feature**, we want to fit a 1D Gaussian:

$$x \sim \mathcal{N}(\mu, \sigma^2)$$

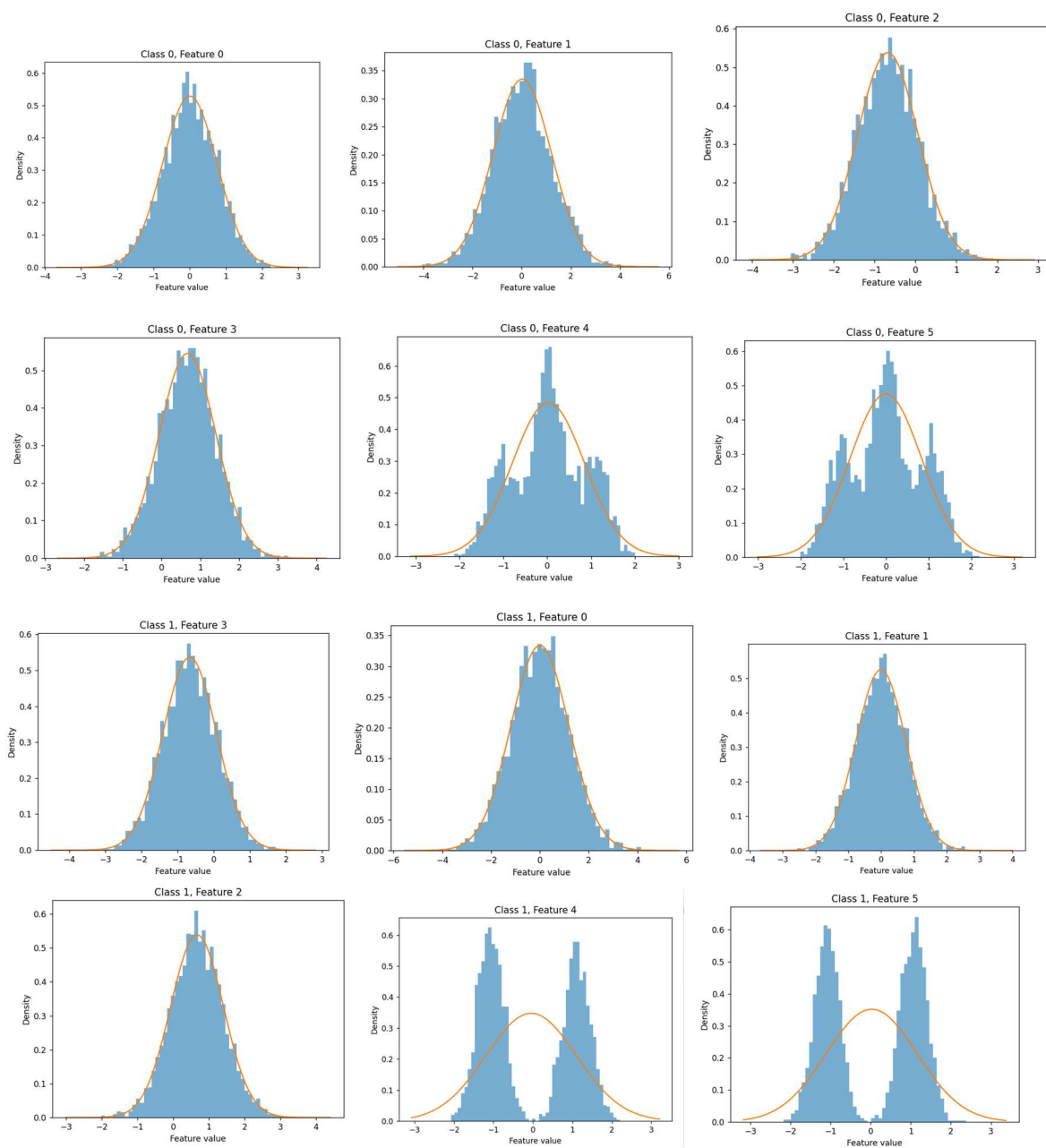
The ML estimates are:

$$\mu_{ML} = \frac{1}{N} \sum_{i=1}^N x_i$$

$$\sigma_{ML}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_{ML})^2$$

Notice: for ML variance we divide by **N**, not by **N-1**.

What do you observe?



The fitted Gaussian densities generally follow the empirical histograms reasonably well for several features. Many distributions appear approximately symmetric, unimodal, and bell-shaped, indicating that a Gaussian assumption is often appropriate. However, some features show deviations from Gaussianity, especially multimodal behavior (features 4 and 5 in both classes).

Are there features for which the Gaussian densities provide a good fit?

Yes. Features 0, 2, and 3 for both classes are well approximated by Gaussian distributions. Their histograms are approximately symmetric and unimodal, and the fitted Gaussian curves closely match the empirical distributions. Feature 1 is also reasonably Gaussian, although the fit is slightly less accurate, especially in the tails.

Are there features for which the Gaussian model seems significantly less accurate?

Yes. Features 4 and 5 show significant deviations from a Gaussian distribution, particularly for class 1. Their histograms exhibit clear bimodal or multimodal behavior, while the fitted Gaussian density is unimodal. As a result, the Gaussian model does not accurately capture the true structure of the data for these features.

MVG

1. Which model performs better?

The best-performing model is **MVG** (Multivariate Gaussian) with an accuracy of about 93.0%. The Naive Bayes Gaussian model achieves a very similar performance (92.8%), while the Tied Gaussian model performs slightly worse (90.7%). MVG performs best because each class has its own covariance matrix: $x | c \sim \mathcal{N}(\mu_c, \Sigma_c)$. This gives the model higher flexibility and produces a quadratic decision boundary. The Tied Gaussian model assumes instead: $\Sigma_0 = \Sigma_1$ which simplifies the classifier to a linear boundary (similar to LDA), reducing modeling capacity. Naive Bayes assumes conditional independence between features, i.e. diagonal covariance matrices. Its performance being very close to MVG suggests that feature correlations are relatively weak.

2. Why is the decision threshold equal to 0?

The classifier uses the log-likelihood ratio:

$$llr(x) = \log \frac{f_{X|C}(x | 1)}{f_{X|C}(x | 0)}$$

With uniform priors:

$$P(C = 1) = P(C = 0) = \frac{1}{2}$$

the Bayes decision rule becomes:

$$llr(x) > -\log \frac{P(C = 1)}{P(C = 0)}$$

Since: $\frac{P(C=1)}{P(C=0)} = 1$

the threshold is: $-\log(1) = 0$

Therefore: predict class 1 if $llr(x) \geq 0$, predict class 0 otherwise.

3. Are covariance values large or small compared to variances?

The covariance matrices show that diagonal elements (variances) are much larger than off-diagonal elements (covariances). This means that each feature mainly varies independently, while interactions between different features are relatively weak. Geometrically, the Gaussian distributions are close to axis-aligned ellipsoids rather than strongly rotated ones (because features are uncorrelated). This is visible from the correlation matrices, where almost all off-diagonal coefficients are between -0.02 and 0.05 .

4. Are the features strongly or weakly correlated?

To analyze feature dependence we compute the **Pearson correlation coefficient**:

$$\text{Corr}(i, j) = \frac{\text{Cov}(i, j)}{\sqrt{\text{Var}(i)\text{Var}(j)}}$$

The correlation matrices show that most off-diagonal values are close to zero. Therefore, **the features are only weakly correlated.**

For example, for the MVG model:

Class 0

$$\begin{bmatrix} 1 & 0.00005 & 0.0327 & 0.0337 \\ 0.00005 & 1 & -0.0178 & -0.0179 \\ 0.0327 & -0.0178 & 1 & -0.0033 \\ 0.0337 & -0.0179 & -0.0033 & 1 \end{bmatrix}$$

Class 1

$$\begin{bmatrix} 1 & -0.0164 & 0.0062 & 0.0173 \\ -0.0164 & 1 & -0.0202 & -0.0161 \\ 0.0062 & -0.0202 & 1 & 0.0489 \\ 0.0173 & -0.0161 & 0.0489 & 1 \end{bmatrix}$$

5. How is this related to the Naive Bayes results?

Naive Bayes assumes conditional independence between features, meaning that covariance matrices are diagonal. Since the actual correlations are already very small, this assumption is approximately satisfied in the dataset. As a result, Naive Bayes loses little information and achieves performance very close to MVG. This explains why Naive Bayes performs almost as well as the full covariance model despite being much simpler.

6. Is the Gaussian assumption accurate for all 6 features?

Not completely. Features 5 and 6 appear to deviate more from the Gaussian assumption and provide weaker discriminative information. When removing features 5 and 6, the error rate slightly decreases, suggesting that these features introduce noise or modeling inaccuracies. However, the degradation is small, meaning that the Gaussian models can still extract some useful information from them.

7. Results using only features 1–2

Using only features 1–2 significantly worsens performance:

MVG $\approx 63.5\%$

Naive Bayes $\approx 63.7\%$

Tied Gaussian $\approx 50.6\%$

These features are weakly discriminative because the class means are very similar while variances differ. Therefore, the classes strongly overlap in feature space, making classification difficult. The tied covariance model performs particularly poorly because it cannot exploit variance differences between classes.

8. Results using only features 3–4

Using only features 3–4 gives much better results:

MVG $\approx 90.6\%$

Tied Gaussian $\approx 90.6\%$

Naive Bayes $\approx 90.6\%$

These features are highly informative because the two classes mainly differ in their means while variances are similar. This creates a clearer separation between classes. Since covariance structures are already similar, the tied covariance assumption does not significantly hurt performance.

9. Is PCA effective?

Applying PCA does not significantly improve performance. The best result is still obtained without dimensionality reduction: MVG with $m = 6$: 93.0% . Accuracy slightly increases as more principal components are retained, indicating that all features contain useful information. Therefore, PCA is not particularly beneficial for this dataset because: the original feature space is already low-dimensional, the dataset does not appear strongly noisy, most features already contribute useful discriminative information.

```
# Media
def compute_mean(D):
    return vcol(D.mean(1))

# Covarianza
def compute_covariance(D, mean):
    Dc = D - mean #centro il dataset
    C = (Dc @ Dc.T) / float(D.shape[1])
    return C

# Covarianza e media per ogni classe
def compute_mean_covariance_per_class_MVG(D, L):
    labelSet = set(L)
    params = {}
    for label in labelSet:
        DX = D[:, L==label]
        mean_x = compute_mean(DX)
        cov_x = compute_covariance(DX, mean_x)
        params[label] = (mean_x, cov_x)
    return params
```

```

def logpdf_GAU_ND(x, mean, cov):
    inv = np.linalg.inv(cov) # inversa della covarianza
    return (
        -0.5 * x.shape[0]*np.log(np.pi*2) \
        -0.5 * np.linalg.slogdet(cov)[1] \
        #slogdet() restituisce (sign, logdet), dove logdet ([1]) ci restituisce il logaritmo del valore assoluto del determinante. Si usa slogdet e non det perché è più stabile
        -0.5 * (((x-mean)) * (inv @ (x-mean))).sum(0) #z.T @ P @ z == (z * (P @ z)).sum()
        # Vogliamo calcolare l'ultimo termine per OGNI CAMPIONE
        # Se x.shape = (4,50) e mean.shape = (4,1)
        #x-mean = (4,50)
        # inv @ (x-mean) = (4,50)
        # (x-mean) * (inv @ (x-mean)) --> (4,50)--> è un prodotto elemento per elemento
        # ma noi vogliamo uno scalare per campione--> (50, ), per questo dobbiamo sommare sulle features (dobbiamo collassare sulle righe)

    )

def logpdf_GAU_global(D, params):
    S = np.zeros((len(params), D.shape[1]))
    for label in range(S.shape[0]):
        mean = params[label][0]
        cov = params[label][1]
        S[label, :] = logpdf_GAU_ND(D, mean, cov)

    return S

def compute_llr(D, params):
    return logpdf_GAU_ND(D, params[1][0], params[1][1]) - logpdf_GAU_ND(D, params[0][0], params[0][1])

def predict_from_llr(llr, threshold = 0):
    predictions = np.zeros(llr.size, dtype = np.int32)
    predictions[llr >= threshold] = 1
    predictions[llr < threshold] = 0

    return predictions

def compute_accuracy(LTE, predictions):
    return (predictions == LTE).sum() / float(LTE.size) * 100

# Naive Bayes
def Naive_Bayes(D, L):
    labelSet = set(L)
    params = {}

```

```

    for label in labelSet:
        DX = D[:, L==label]
        mean_x = compute_mean(DX)
        cov_x = compute_covariance(DX, mean_x)
        params[label] = (mean_x, (cov_x * np.eye(D.shape[0]))) #per avere ma-
trice diagonale

    return params

def tied_covariance(D,L):
    params = {}
    labelSet = set(L)
    cov = 0
    means = {}
    for label in labelSet:
        DX = D[:, L == label]
        mean_x = compute_mean(DX)
        cov_x = compute_covariance(DX, mean_x)
        cov += cov_x * DX.shape[1]
        means[label] = mean_x

    C_global = cov / D.shape[1]
    for label in labelSet:
        params[label] = (means[label], C_global)

    return params

def compute_correlation_Pearson(C):
    diag = C.diagonal()
    std = np.sqrt(diag)
    Corr = C / (vcol(std) @ vrow(std))

    return Corr

```

Bayes error risk

1. Effective prior

The five applications can be rewritten using the effective prior: $\tilde{\pi} = \frac{\pi_1 C_{fn}}{\pi_1 C_{fn} + (1 - \pi_1) C_{fp}}$

The results are: (π_1, C_{fn}, C_{fp})

$(0.5, 1, 1) \rightarrow \tilde{\pi} = 0.5$
 $(0.9, 1, 1) \rightarrow \tilde{\pi} = 0.9$
 $(0.1, 1, 1) \rightarrow \tilde{\pi} = 0.1$
 $(0.5, 1, 9) \rightarrow \tilde{\pi} = 0.1$
 $(0.5, 9, 1) \rightarrow \tilde{\pi} = 0.9$

So, higher false positive cost corresponds to a lower effective prior, meaning stronger security. Higher false negative cost corresponds to a higher effective prior, meaning more tolerance toward accepting legit users.

2. Actual DCF and minimum DCF

The three models were evaluated for the three effective priors 0.1,0.5,0.9.

Model	$\tilde{\pi}$	actualDCF	minDCF
MVG	0.1	0.305	0.263
MVG	0.5	0.140	0.130
MVG	0.9	0.400	0.342
Tied	0.1	0.415	0.369
Tied	0.5	0.190	0.181
Tied	0.9	0.477	0.442
Naive Bayes	0.1	0.302	0.257
Naive Bayes	0.5	0.144	0.131
Naive Bayes	0.9	0.389	0.351

3. Which models perform best?

The best models are **MVG** and **Naive Bayes**.

In terms of **minDCF**:

for $\tilde{\pi} = 0.1$, Naive Bayes is slightly better: 0.257 vs 0.263;

for $\tilde{\pi} = 0.5$, MVG and Naive Bayes are almost identical: 0.130 vs 0.131;

for $\tilde{\pi} = 0.9$, MVG is slightly better: 0.342 vs 0.351.

The Tied Gaussian model is always worse.

4. Are the relative results consistent?

Yes. The ranking is consistent across the applications:

$$\text{MVG} \approx \text{Naive Bayes} < \text{Tied Gaussian}$$

where lower DCF means better performance. This means that MVG and Naive Bayes have similar separation capability, while Tied Covariance is consistently less effective.

5. Are the models well calibrated?

Calibration can be evaluated by comparing actualDCF and minDCF.

A well-calibrated model has: $\text{actualDCF} \approx \text{minDCF}$

MVG and Naive Bayes are reasonably well calibrated, especially for $\tilde{\pi} = 0.5$.

For example:

$$MVG: 0.140 - 0.130 = 0.010$$

$$Naive: 0.144 - 0.131 = 0.013$$

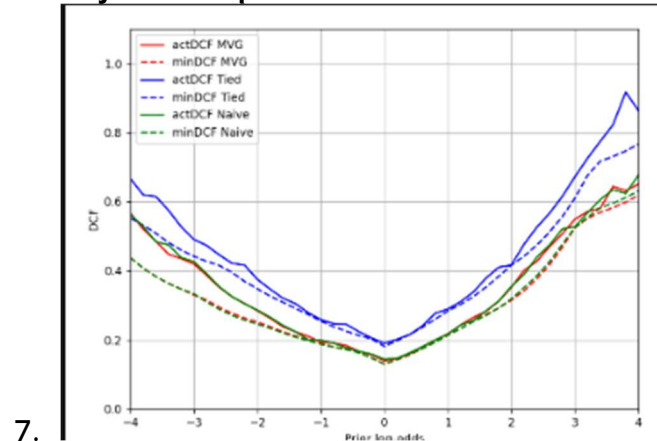
For extreme priors, the gap increases:

$$MVG, \tilde{\pi} = 0.9: 0.400 - 0.342 = 0.058$$

$$Naive, \tilde{\pi} = 0.9: 0.389 - 0.351 = 0.038$$

The Tied model is less calibrated and has higher DCF values overall.

6. Bayes error plot



The Bayes error plot confirms the previous results. The MVG and Naive Bayes curves are very close, both for actualDCF and minDCF. The Tied Gaussian curve is higher over most of the prior log-odds range, meaning worse performance. The minimum DCF curves show that the model ranking is mostly consistent: $MVG \approx Naive Bayes < Tied$.

The actualDCF curves show that MVG and Naive Bayes are better calibrated than Tied, because their actualDCF is closer to minDCF.

EVALUATE SCORES:

```
def evaluateScores(S, L, prior=0.1, Cfn=1.0, Cfp=1.0):
```

```
    return compute_minDCF_binary_fast(prior, Cfn, Cfp, L, S)
```

```
def compute_confusion_matrix(predictedLabels, classLabels):
    n_classes = classLabels.max() + 1
    M = np.zeros((n_classes, n_classes), dtype = np.int32)
    for i in range(classLabels.size):
        M[predictedLabels[i], classLabels[i]] += 1

    return M

def compute_dcf(predictedLabels, classLabels, prior, Cfn, Cfp):
    M = compute_confusion_matrix(predictedLabels, classLabels)

    Pfn = M[0,1] / (M[0,1] + M[1,1])
```

```

Pfp = M[1,0] / (M[0,0] + M[1,0])

dcf = prior * Cfn * Pfn + (1 - prior) * Cfp * Pfp

return dcf

def compute_normalized_dcf(prior, Cfn, Cfp, dcf):
    den = float(np.minimum(prior * Cfn, (1- prior) * Cfp))

    return dcf/den

def compute_actual_DCF_fast(prior, Cfn, Cfp, llr, labels):
    pred = compute_optimal_Bayes_binary(llr, prior, Cfn, Cfp)
    dcf = compute_dcf(pred, labels, prior, Cfn, Cfp)
    dcf_norm = compute_normalized_dcf(prior, Cfn, Cfp, dcf)

    return dcf_norm

def compute_all_thresholds_fast(llr, labels):
    # Ordino gli score dal più piccolo al più grande
    idx = np.argsort(llr)
    llr_sorted = llr[idx]
    labels_sorted = labels[idx]

    # Numero tot di campioni veri della classe True e della classe False
    nTrue = (labels_sorted == 1).sum()
    nFalse = (labels_sorted == 0).sum()

    # soglia iniziale -inf--> tutti predetti come classe 1
    nFN = 0
    nFP = nFalse

    Pfn = []
    Pfp = []

    # Salvo punto di partenza
    Pfn.append(nFN / nTrue)
    Pfp.append(nFP / nFalse)

    # Simuliamo spostamento soglia da sinistra verso destra
    for i in range(len(llr_sorted)):
        if labels_sorted[i] ==1:
            nFN += 1
        else :
            nFP -= 1

    # Salvo i nuovi error rates per le classi
    Pfn.append(nFN / nTrue)

```

```

        Pfp.append(nFP / nFalse)

    Pfn_out = []
    Pfp_out = []

    llr_sorted_exit = np.concatenate([-np.array([np.inf]), llr_sorted])

    for i in range(len(llr_sorted_exit)):

        if i==len(llr_sorted_exit) -1 or llr_sorted_exit[i+1] !=
llr_sorted_exit[i]:
            Pfn_out.append(Pfn[i])
            Pfp_out.append(Pfp[i]) # salviamo solo se effettivamente c'è un
cambio di soglia

    return np.array(Pfn_out), np.array(Pfp_out)

def compute_minDCF_binary_fast(prior, Cfn, Cfp, labels, llr):
    Pfn, Pfp = compute_all_thresholds_fast(llr, labels)

    dcf = prior * Cfn * Pfn + (1-prior) * Cfp * Pfp
    dcfNorm = compute_normalized_dcf(prior, Cfn, Cfp, dcf)

    return np.min(dcfNorm)

```

BINARY TIED train():

```

def train_Tied(DTR, LTR):
    params = {}
    C_tied = np.zeros((DTR.shape[0], DTR.shape[0]))

    for c in [0,1]:
        Dc = DTR[:, LTR == c]
        mean,C = compute_mean_c(Dc)
        params[c] = (mean, None)

        C_tied += C * Dc.shape[1]

    C_tied = C_tied / DTR.shape[1]

    params[0] = (params[0][0], C_tied)
    params[1] = (params[1][0], C_tied)

    return params

def compute_llr_Tied(DTE, params):
    mean0, C = params[0]
    mean1, C = params[1]

```

```

l10 = logpdf_GAU_ND(DTE, mean0, C)
l11 = logpdf_GAU_ND(DTE, mean1, C)

return l11 - l10

```

BINARY NAÏVE

```

def train_Naive(DTR, LTR):
    params = {}

    for i in [0,1]:
        Dc = DTR[:, LTR == i]
        mean, C = compute_mean_c(Dc)

        C_diag = C * np.eye(C.shape[0])

        params[i] = (mean, C_diag)

    return params

def compute_llr_Naive(DTE, params):
    mean0, C0 = params[0]
    mean1, C1 = params[1]

    l10 = logpdf_GAU_ND(DTE, mean0, C0)
    l11 = logpdf_GAU_ND(DTE, mean1, C1)

    return l11 - l10

```

```

def compute_optimal_Bayes_binary(llr, prior, Cfn, Cfp):
    th = -np.log((prior * Cfn) / ((1-prior) * Cfp))

    return np.int32(llr > th)

```

MVG:

```

def trainMVG (DTR, LTR):
    params = {}
    for c in [0,1]:
        Dc = DTR[:, LTR == c]
        mean, C = compute_mean_c(Dc)
        params[c] = (mean, C)

    return params

def compute_llr_MVG(DTE, params):
    mean0, C0 = params[0]
    mean1, C1 = params[1]

```

```

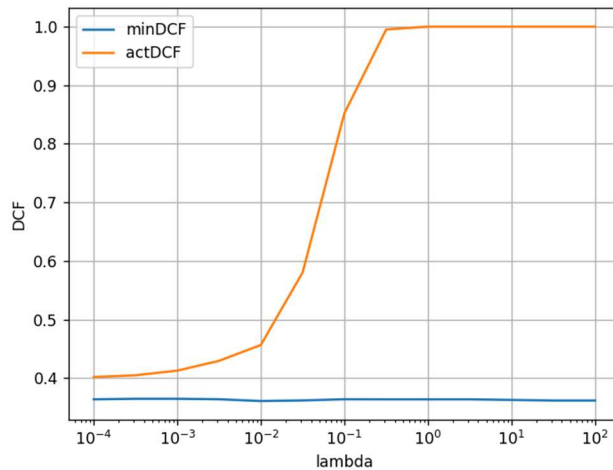
110 = logpdf_GAU_ND(DTE, mean0, C0)
111 = logpdf_GAU_ND(DTE, mean1, C1)

return 111 - 110

```

Logistic Regression

- Standard Logistic Regression Binary



The standard binary logistic regression model was trained using different values of the regularization parameter λ , evaluating both minimum DCF and actual DCF for the target application $\pi_T = 0.1$.

The obtained results are:

λ	minDCF	actDCF
$3.16 \cdot 10^{-4}$	0.3650	0.4051
10^{-3}	0.3650	0.4130
$3.16 \cdot 10^{-3}$	0.3641	0.4297
10^{-2}	0.3611	0.4568
$3.16 \cdot 10^{-2}$	0.3621	0.5805
10^{-1}	0.3641	0.8522
$3.16 \cdot 10^{-1}$	0.3640	0.9950
1	0.3640	1.0000

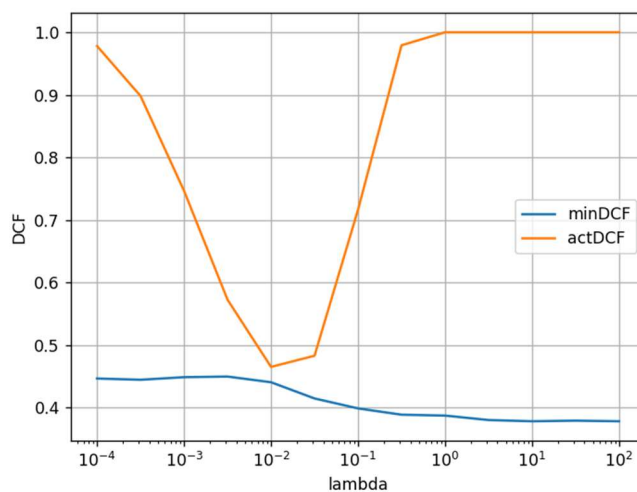
The minimum DCF remains almost constant around 0.36 for all values of λ . This means that the discriminative capability of the logistic regression classifier is not significantly affected by the regularization parameter when the full training set is used.

On the contrary, actualDCF increases significantly as λ grows. For large regularization values, the model weights are strongly penalized and the scores become compressed. As a consequence, the probabilistic interpretation of the scores degrades and the classifier becomes poorly calibrated. Therefore:

- **minDCF** measures the best achievable discrimination capability, independently of score calibration;
- **actDCF** measures the actual application performance using the Bayes threshold and is strongly affected by calibration quality.

Since the dataset contains many training samples, overfitting is limited even for very small regularization values. In this scenario, regularization is not particularly beneficial and excessive regularization mainly causes underfitting and calibration degradation. Lower values of λ correspond to lower regularization and higher risk of overfitting, whereas larger values reduce overfitting but may excessively constrain the model parameters, reducing score quality and probabilistic interpretability.

2. Logistic Regression with reduced training set



The experiment was repeated using only 1 sample every 50 training samples. In this case, the effect of regularization becomes much more visible.

For very small λ :

λ	minDCF	actDCF
10^{-4}	0.447	0.978
10^{-3}	0.449	0.747

The model strongly overfits the small training set: the weights become excessively large, the scores become unstable, actDCF becomes extremely poor.

For intermediate values of λ :

λ	minDCF	actDCF
10^{-2}	0.441	0.465
$3.16 \cdot 10^{-2}$	0.415	0.483

regularization improves generalization and stabilizes the classifier.

For very large λ :

λ	minDCF	actDCF
1	0.387	1.000
10	0.378	1.000

the model underfits because the weights are excessively penalized.

Therefore:

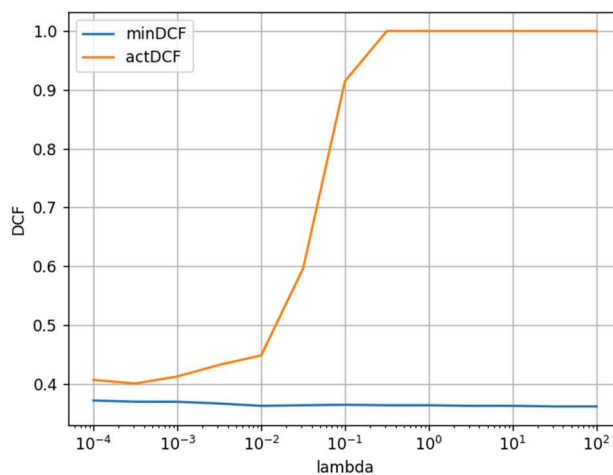
small $\lambda \rightarrow$ overfitting (big actDCF),

intermediate $\lambda \rightarrow$ best tradeoff,

large $\lambda \rightarrow$ underfitting.

The minDCF curve changes less because it always searches for the optimal threshold, even if the scores are poorly calibrated.

4. Prior-weighted Logistic Regression



The weighted logistic regression model was trained using the target prior: $\pi_T = 0.1$ (incorporated in the scores).

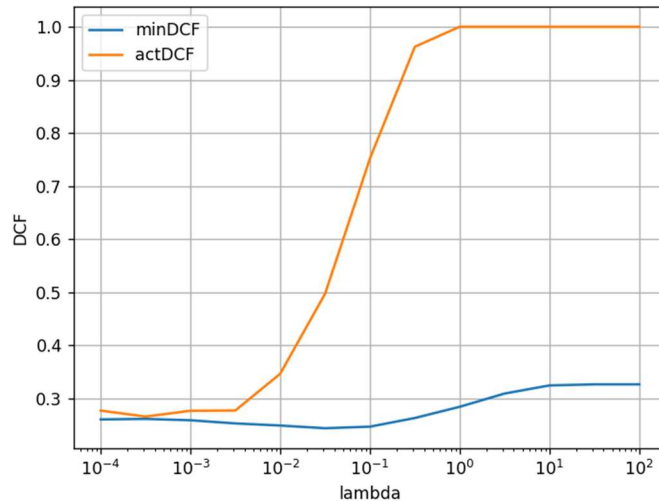
The obtained results are very similar to the standard logistic regression model.

For example:

λ	minDCF	actDCF
10^{-2}	0.363	0.449
10^{-1}	0.365	0.915
1	0.364	1.000

No significant improvements can be observed. Both minDCF and actDCF behave similarly to the standard model. This suggests that the mismatch between empirical training prior and target prior is not the main limitation of the classifier. The main limitation appears to be the linear nature of the model itself.

5. Quadratic Logistic Regression



Quadratic logistic regression was implemented by expanding the feature space with quadratic terms. The quadratic model significantly improves performance:

Model	minDCF
Linear Logistic Regression	≈ 0.36
Quadratic Logistic Regression	$\approx 0.24 - 0.26$

The quadratic model achieves the best discriminative capability among all tested logistic regression models. This indicates that the classes are not well separable using only a linear decision boundary. By introducing quadratic features, the classifier can learn non-linear separation rules in the original feature space. The best performance is obtained for small/intermediate values of λ . For large λ , performance degrades because strong regularization prevents the model from exploiting the additional quadratic features. As in the previous experiments, actDCF worsens much faster than minDCF for large regularization values, confirming that regularization negatively affects calibration.

5. Comparison with Gaussian Models

The obtained results are:

Model	minDCF	actDCF
MVG	0.263	0.305
Tied Gaussian	0.363	0.406
Naive Bayes	0.257	0.302
Linear Logistic Regression	≈ 0.36	≈ 0.40
Weighted Logistic Regression	≈ 0.36	≈ 0.40
Quadratic Logistic Regression	≈ 0.244	≈ 0.266

The best model is Quadratic Logistic Regression, since it achieves the lowest minDCF and also a very good actualDCF. This suggests that: the classes are not linearly separable, non-linear decision boundaries are beneficial, quadratic feature expansion captures important interactions between features. Naive Bayes and MVG also achieve strong performance, suggesting that Gaussian assumptions approximately hold and that feature

correlations are relatively weak. The Tied Gaussian model performs worse because forcing the same covariance matrix for both classes reduces modeling flexibility.

```
# compute matrix of posteriors from class-conditional log-likelihoods (each
column represents a sample) and prior array
def compute_posteriors(log_clas_conditional_ll, prior_array):
    logJoint = log_clas_conditional_ll + vcol(numpy.log(prior_array))
    logPost = logJoint - scipy.special.logsumexp(logJoint, 0)
    return numpy.exp(logPost)

# Compute optimal Bayes decisions for the matrix of class posterior (each col-
umn refers to a sample)
def compute_optimal_Bayes(posterior, costMatrix):
    expectedCosts = costMatrix @ posterior
    return numpy.argmin(expectedCosts, 0)

# Build uniform cost matrix with cost 1 for all kinds of error, and cost 0 for
correct assignments
def uniform_cost_matrix(nClasses):
    return numpy.ones((nClasses, nClasses)) - numpy.eye(nClasses)

# Assume that classes are labeled 0, 1, 2 ... (nClasses - 1)
def compute_confusion_matrix(predictedLabels, classLabels):
    nClasses = classLabels.max() + 1
    M = numpy.zeros((nClasses, nClasses), dtype=numpy.int32)
    for i in range(classLabels.size):
        M[predictedLabels[i], classLabels[i]] += 1
    return M

# Optimal Bayes decisions for binary tasks with log-likelihood-ratio scores
def compute_optimal_Bayes_binary_llr(llr, prior, Cfn, Cfp):
    th = -numpy.log( (prior * Cfn) / ((1 - prior) * Cfp) )
    return numpy.int32(llr > th)

# Multiclass solution that works also for binary problems
def compute_empirical_Bayes_risk(predictedLabels, classLabels, prior_array,
costMatrix, normalize=True):
    M = compute_confusion_matrix(predictedLabels, classLabels) # Confusion ma-
trix
    errorRates = M / vrow(M.sum(0))
    bayesError = ((errorRates * costMatrix).sum(0) * prior_ar-
ray.ravel()).sum()
    if normalize:
        return bayesError / numpy.min(costMatrix @ vcol(prior_array))
    return bayesError

# Specialized function for binary problems (empirical_Bayes_risk is also
called DCF or actDCF)
```

```

def compute_empirical_Bayes_risk_binary(predictedLabels, classLabels, prior,
Cfn, Cfp, normalize=True):
    M = compute_confusion_matrix(predictedLabels, classLabels) # Confusion ma-
    trix
    Pfn = M[0,1] / (M[0,1] + M[1,1])
    Pfp = M[1,0] / (M[0,0] + M[1,0])
    bayesError = prior * Cfn * Pfn + (1-prior) * Cfp * Pfp
    if normalize:
        return bayesError / numpy.minimum(prior * Cfn, (1-prior)*Cfp)
    return bayesError

# Compute empirical Bayes (DCF or actDCF) risk from llr with optimal Bayes de-
# cisions
def compute_empirical_Bayes_risk_binary_llr_optimal_decisions(llr,
classLabels, prior, Cfn, Cfp, normalize=True):
    predictedLabels = compute_optimal_Bayes_binary_llr(llr, prior, Cfn, Cfp)
    return compute_empirical_Bayes_risk_binary(predictedLabels, classLabels,
prior, Cfn, Cfp, normalize=normalize)

# Compute minDCF (fast version)
# If we sort the scores, then, as we sweep the scores, we can have that at
# most one prediction changes everytime. We can then keep a running confusion
# matrix (or simply the number of false positives and false negatives) that is
# updated everytime we move the threshold

# Auxiliary function, returns all combinations of Pfp, Pfn corresponding to
# all possible thresholds
# We do not consider -inf as threshld, since we use as assignment llr > th, so
# the left-most score corresponds to all samples assigned to class 1 already
def compute_Pfn_Pfp_allThresholds_fast(llr, classLabels):
    llrSorter = numpy.argsort(llr)
    llrSorted = llr[llrSorter] # We sort the llrs
    classLabelsSorted = classLabels[llrSorter] # we sort the labels so that
    they are aligned to the llrs

    Pfp = []
    Pfn = []

    nTrue = (classLabelsSorted==1).sum()
    nFalse = (classLabelsSorted==0).sum()
    nFalseNegative = 0 # With the left-most theshold all samples are assigned
    to class 1
    nFalsePositive = nFalse

    Pfn.append(nFalseNegative / nTrue)
    Pfp.append(nFalsePositive / nFalse)

    for idx in range(len(llrSorted)):

```

```

        if classLabelsSorted[idx] == 1:
            nFalseNegative += 1 # Increasing the threshold we change the as-
            signment for this llr from 1 to 0, so we increase the error rate
        if classLabelsSorted[idx] == 0:
            nFalsePositive -= 1 # Increasing the threshold we change the as-
            signment for this llr from 1 to 0, so we decrease the error rate
        Pfn.append(nFalseNegative / nTrue)
        Pfp.append(nFalsePositive / nFalse)

    #The last values of Pfn and Pfp should be 1.0 and 0.0, respectively
    #Pfn.append(1.0) # Corresponds to the numpy.inf threshold, all samples are
assigned to class 0
    #Pfp.append(0.0) # Corresponds to the numpy.inf threshold, all samples are
assigned to class 0
    llrSorted = numpy.concatenate([-numpy.array([numpy.inf]), llrSorted])

    # In case of repeated scores, we need to "compact" the Pfn and Pfp arrays
    (i.e., we need to keep only the value that corresponds to an actual change of
    the threshold
    PfnOut = []
    PfpOut = []
    thresholdsOut = []
    for idx in range(len(llrSorted)):
        if idx == len(llrSorted) - 1 or llrSorted[idx+1] != llrSorted[idx]: #
We are indeed changing the threshold, or we have reached the end of the array
of sorted scores
            PfnOut.append(Pfn[idx])
            PfpOut.append(Pfp[idx])
            thresholdsOut.append(llrSorted[idx])

    return numpy.array(PfnOut), numpy.array(PfpOut), numpy.array(threshold-
sOut) # we return also the corresponding thresholds

# Note: for minDCF llrs can be arbitrary scores, since we are optimizing the
threshold
# We can therefore directly pass the logistic regression scores, or the SVM
scores
def compute_minDCF_binary_fast(llr, classLabels, prior, Cfn, Cfp, return-
Threshold=False):

    Pfn, Pfp, th = compute_Pfn_Pfp_allThresholds_fast(llr, classLabels)
    minDCF = (prior * Cfn * Pfn + (1 - prior) * Cfp * Pfp) / numpy.mini-
mum(prior * Cfn, (1-prior)*Cfp) # We exploit broadcasting to compute all DCFs
for all thresholds
    idx = numpy.argmin(minDCF)
    if returnThreshold:
        return minDCF[idx], th[idx]
    else:

```

```
return minDCF[idx]
```

```
compute_actDCF_binary_fast = compute_empirical_Bayes_risk_binary_llr_optimal_decisions # To have a function with a similar name to the minDCF one
```

```
def trainLogRegBinary(DTR, LTR, l): #takes training samples DTR, labels LTR, and lambda (regularization)
    ZTR = LTR * 2.0 - 1.0 # We convert labels from {0,1} to {-1,1}
    # We define the objective function
    def logreg_obj_with_grad(v): # Il parametro è solo 1, v, perché scipy.optimize si aspetta f(x) con un singolo parametro
        # Impacchettiamo w,b in v= [w,b]
        # Spacchettiamo v in w e b
        w = v[0:-1]
        b = v[-1]
        s = np.dot(vcol(w).T, DTR).ravel() + b #s = w.T @ DTR + b
        # ravel() serve per trasformare il risultato in un array monodimensionale
        # In dataset Iris:
        # DTR appartiene a  $R^{4 \times N}$ , 4 features e N samples
        # w appartiene a  $R^4$ , b è uno scalare, con vcol() diventa 4x1, trasposto 1x4
        #  $(1 \times 4) (4 \times N) = 1 \times N \rightarrow s = [s_1, s_2, \dots, s_N]$  otteniamo gli score di tutti i sample contemporaneamente
        # con ravel() da  $(1, N)$  otteniamo  $(N,)$ 
        loss = np.logaddexp(0, -ZTR * s)
        # Adesso calcoliamo il gradiente:
        G = -ZTR / (1.0 + np.exp(ZTR * s))

        # Gradiente rispetto a w
        Gw = (vrow(G) * DTR).mean(1) + l * w.ravel()

        # Gradiente rispetto a b
        Gb = G.mean() # G è un array di dimensione N, con G.mean() otteniamo la media di tutti i campioni

        return loss.mean() + l / 2 * np.linalg.norm(w)**2, np.hstack([Gw, np.array(Gb)])
    # con np.hstack ricomponiamo il vettore di derivate parziali
    # Ottimizzazione
    vf = scipy.optimize.fmin_l_bfgs_b(
        func=logreg_obj_with_grad,
        x0=np.zeros(DTR.shape[0]+1) # DTR.shape[0] è il numero di features, +1 per il bias
    )[0]
    w = vf[0:-1]
    b = vf[-1]
```

```

    return w, b

# Loss pesata con la prior applicativa
def trainWeightedLogRegBinary(DTR, LTR, l, pT):
    ZTR = LTR * 2.0 - 1.0 # We convert labels from {0,1} to {-1,1}
    # Calcoliamo il peso delle 2 classi
    wTrue = pT / (ZTR > 0).sum()
    wFalse = (1-pT) / (ZTR < 0).sum()
    # We define the objective function
    def logreg_obj_with_grad(v):
        w = v[0:-1]
        b = v[-1]
        s = np.dot(vcol(w).T, DTR).ravel() + b
        loss = np.logaddexp(0, -ZTR * s)
        # Appliciamo i pesi alla loss
        loss[ZTR > 0] *= wTrue
        loss[ZTR < 0] *= wFalse
        # Adesso calcoliamo il gradiente:
        G = -ZTR / (1.0 + np.exp(ZTR * s))
        # Appliciamo i pesi al gradiente
        G[ZTR > 0] *= wTrue
        G[ZTR < 0] *= wFalse

        Gw = (vrow(G) * DTR).sum(1) + l * w.ravel()
        Gb = G.sum()

        return loss.sum() + l / 2 * np.linalg.norm(w)**2, np.hstack([Gw,
np.array(Gb)])

    vf = scipy.optimize.fmin_l_bfgs_b(
        func=logreg_obj_with_grad,
        x0=np.zeros(DTR.shape[0]+1)
    )[0]
    w = vf[0:-1]
    b = vf[-1]
    return w, b

def expand_features_quadratic(D):
    expanded = []

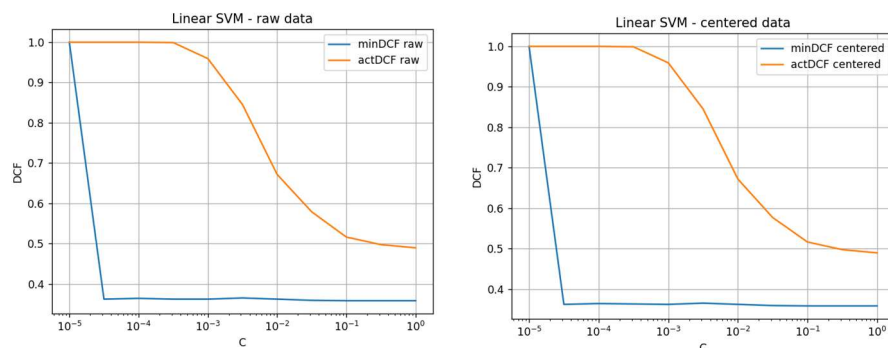
    for i in range(D.shape[1]):
        x = D[:, i:i+1]
        xxT = x @ x.T
        phi = np.vstack([xxT.reshape((-1, 1)), x]) #concatena le 36 features
        #espanse con le features originali
        expanded.append(phi)

    return np.hstack(expanded)

```

SVM

6. Apply the Linear SVM to the project data. Train the model with different values of C . What do you observe? Does the regularization coefficient significantly affect the results?



We trained a Linear SVM with: $K = 1$, $\pi_T = 0.1$, $C \in [10^{-5}, 10^0]$

The parameter C controls the trade-off between margin maximization and training errors:

small $C \rightarrow$ strong regularization

large $C \rightarrow$ weak regularization

For very small values of C :

C	minDCF	actDCF
10^{-5}	1.000	1.000

the classifier strongly underfits because the regularization is excessive. As C increases, the performance improves significantly. Starting from approximately: $C \geq 10^{-4}$ the minimum DCF stabilizes around: **minDCF ≈ 0.36** . This suggests that the Linear SVM is relatively robust to the choice of C once the regularization is not excessively strong.

2. Are the scores well calibrated for the target application?

The scores are not perfectly calibrated. Even for large values of C :

C	minDCF	actDCF
1	0.358	0.489

we observe: $\text{actDCF} > \text{minDCF}$

This means that the SVM scores are discriminative but not well calibrated. Indeed, SVM outputs are not true log-likelihood ratios, therefore the Bayes threshold is not optimal for the produced scores. The model achieves reasonable ranking capability (low minDCF), but poorer probabilistic interpretation (higher actDCF).

3. What can we conclude on Linear SVM? How does it compare to other linear models?

Linear SVM achieves: $\text{minDCF} \approx 0.36$

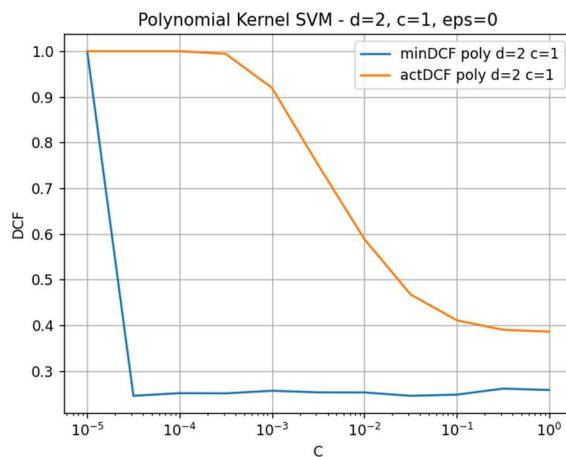
which is very similar to standard Logistic Regression: Logistic Regression $\text{minDCF} \approx 0.36$.

Therefore, both linear discriminative models provide comparable performance. The results suggest that the dataset is not linearly separable and that linear decision boundaries are insufficient to fully capture the structure of the data.

4. Repeat the analysis with centered data. Are the results significantly different?

The centered and non-centered versions of Linear SVM produce almost identical curves: same minDCF values, very similar actDCF values. This behavior is expected because the Linear SVM already includes a bias term: $s(x) = w^T x + b$ which partially compensates for global shifts in the feature space. Therefore, centering does not significantly affect the classifier performance for this dataset.

5. Apply the Polynomial Kernel SVM ($d = 2, c = 1, \epsilon = 0$). What do you observe?



We trained a Polynomial Kernel SVM with: degree $d = 2$, $c = 1$, $\epsilon = 0$ using the original non-centered features. As for Linear SVM, very small values of C produce severe underfitting:

C	minDCF	actDCF
10^{-5}	1.000	1.000

As C increases, performance improves significantly. Starting from approximately: $C \geq 10^{-4}$ the minimum DCF stabilizes around: $\text{minDCF} \approx 0.25$. The Polynomial Kernel SVM therefore achieves substantially better performance than the Linear SVM.

6. Are the results consistent with previous models (Logistic Regression and MVG)?

Yes. The improvement obtained with the quadratic kernel is consistent with the previous experiments:

Model	minDCF
Linear Logistic Regression	≈ 0.36
Linear SVM	≈ 0.36

Quadratic Logistic Regression	$\approx 0.24 - 0.26$
Polynomial SVM $d = 2$	≈ 0.25

These results indicate that the dataset contains important non-linear feature interactions. Quadratic models can learn curved decision boundaries and therefore better separate the classes.

7. What about actDCF for the Polynomial Kernel SVM?

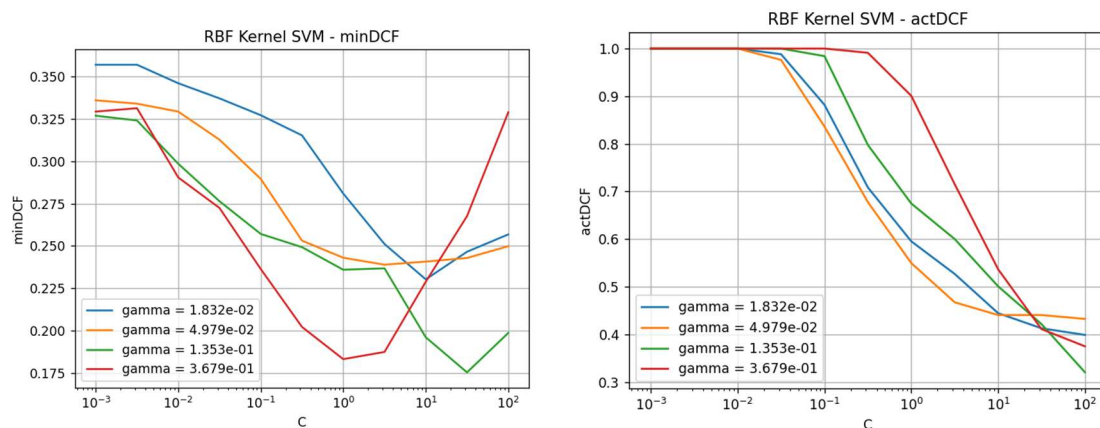
Although minDCF improves significantly, we still observe: $\text{actDCF} > \text{minDCF}$

For large values of C :

minDCF	actDCF
0.25	0.39

Therefore, the scores are still not perfectly calibrated. However, the calibration gap is smaller than for Linear SVM, suggesting moderately improved score quality.

7. Apply the RBF Kernel SVM. Are there values of γ and C that provide better results?



We evaluated the RBF Kernel SVM using: $\epsilon = 1$, $\pi_T = 0.1$, $C \in [10^{-3}, 10^2]$, $\gamma \in \{e^{-4}, e^{-3}, e^{-2}, e^{-1}\}$. For all values of γ , very small values of C produce severe underfitting:

minDCF	actDCF
0.33 – 0.36	1.0

As C increases, performance improves significantly. The best configuration is obtained for:

$\gamma = e^{-1} \approx 0.3679$ with approximately: $C \approx 1$ achieving: **best minDCF ≈ 0.18**

This is the best result among all tested SVM models.

9. How do the RBF results compare to previous models?

Comparison of best minDCF values:

Model	Best minDCF
-------	-------------

Linear SVM	≈ 0.36
Polynomial SVM $d = 2$	≈ 0.25
RBF SVM	≈ 0.18

The RBF Kernel SVM significantly outperforms all previous models. This strong improvement suggests that the dataset contains highly non-linear structures that cannot be modeled effectively using linear or simple quadratic decision boundaries.

10. Are the RBF scores well calibrated?

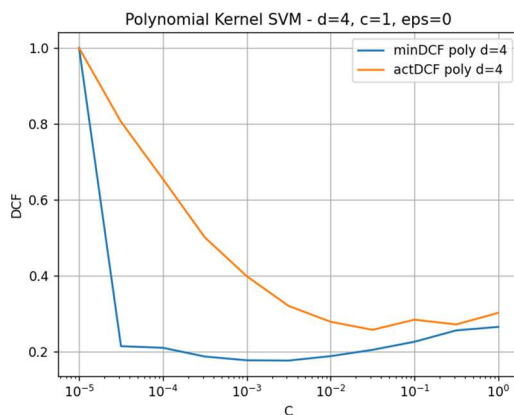
Even in the RBF case we observe: $\text{actDCF} > \text{minDCF}$ showing that the scores are not perfectly calibrated. However, for the best configurations:

minDCF	actDCF
0.18	0.32 – 0.38

the calibration gap is smaller than in simpler models. Therefore, the RBF Kernel SVM produces much stronger discriminative scores, although they are still not fully calibrated.

11. Optional: Polynomial Kernel with degree $d = 4$

We also evaluated a Polynomial Kernel SVM with: degree $d = 4$, $c = 1$, $\epsilon = 0$

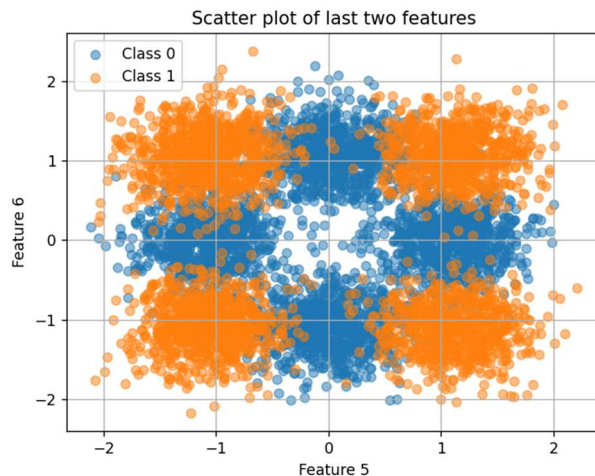


The degree-4 kernel achieves better performance than the degree-2 kernel:

Model	Best minDCF
Polynomial SVM $d = 2$	≈ 0.25
Polynomial SVM $d = 4$	≈ 0.18

The actDCF also improves considerably: $\text{actDCF} \approx 0.26 - 0.30$ for intermediate values of C . The best results are obtained for moderate regularization values, while: very small C causes underfitting, very large C starts degrading minDCF, suggesting mild overfitting.

12. Why does the degree-4 kernel work better?



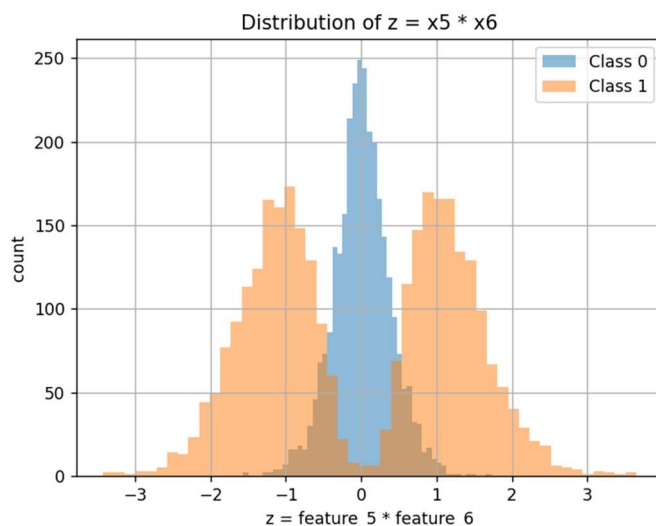
The scatter plot of the last two features reveals a highly non-linear structure. The two classes form multiple separated regions arranged approximately in a checkerboard/cross-like configuration: class 0 occupies central regions, class 1 occupies outer/corner regions.

Such structures cannot be effectively separated using linear boundaries and are difficult to model using only quadratic decision functions.

We also analyzed the transformed variable:

$$z_i = y_i[0] \cdot y_i[1]$$

where $y_i = x_i[4:6]$.



The histogram shows:

class 0 concentrated around $z \approx 0$,

class 1 distributed in outer positive and negative regions.

This distribution cannot be separated using a single linear threshold, whereas higher-order polynomial kernels can naturally model interval-based and multi-region decision boundaries.

Overall, the experiments strongly suggest that the project dataset is highly non-linear and benefits

significantly from flexible kernel-based classifiers.

TRAIN A LINEAR SVM WITH THE DUAL PROBLEM:

It returns : $w, b \rightarrow s(x) = w^T x + b$

```
def train_dual_SVM_linear(DTR, LTR, C, K=1):    #C = coefficiente di regolar-
izzazione, K = parametro usato per bias regolarizzato
    ZTR = LTR * 2.0 - 1.0    //conversion labels (-1,1)
    # Incorporiamo il bias nei pesi
```

```

# Vogliamo trasformare: score = w^T x + b
# in : score = w_hat^T x_hat
# dove: x_hat è un vettore colonna (x,K)
DTR_EXT = np.vstack([DTR, np.ones((1, DTR.shape[1])) * K])
# Dobbiamo implementare questa matrice:  $H_{ij} = z_i z_j x_i^T x_j$ 
G_hat = DTR_EXT.T @ DTR_EXT * x_i^T x_j
# vcol(ZTR) * vrow(ZTR) crea tutte le combinazioni z_i z_j.
H = G_hat * vcol(ZTR) * vrow(ZTR)

```

L'SVM duale massimizza: $J_D(\alpha) = -\frac{1}{2}\alpha^T H \alpha + \alpha^T 1$, ma scipy minimizza quindi:

```

def dual_objective(alpha):
    H_alpha = H @ vcol(alpha)
    loss = 0.5 * (vrow(alpha) @ H_alpha).ravel() - alpha.sum()
    # Calcola il gradiente ottenuto tramite derivata: H alpha - 1
    grad = H_alpha.ravel() - np.ones(alpha.size)

    return loss, grad

# 0 <= alpha_i <= C, Questi sono vincoli box, gestibili da L-BFGS-B.
alpha0 = np.zeros(DTR_EXT.shape[1])
bounds = [(0, C) for _ in LTR]
alphaStar, _, _ = scipy.optimize.fmin_l_bfgs_b(
    dual_objective, alpha0, bounds=bounds, factr=np.nan, pgtol=1e-5)
Relazione primale - duale:  $\hat{w} = \sum_i \alpha_i z_i \hat{x}_i$ 
w_hat = (vrow(alphaStar) * vrow(ZTR) * DTR_EXT).sum(1)
# w_hat contiene: w_hat = vcol([w], [b/K]), dobbiamo separare w,b
w = w_hat[0:DTR.shape[0]]
b = w_hat[-1] * K

```

Primal loss: $J(\hat{w}) = \frac{1}{2} \|\hat{w}\|^2 + C \sum_i \max(0, 1 - z_i \text{score}_i)$

```

def primal_objective(w_hat):
    scores = (vrow(w_hat) @ DTR_EXT).ravel()
    hinge_losses = np.maximum(0, 1 - ZTR * scores)
    return 0.5 * np.linalg.norm(w_hat) ** 2 + C * hinge_losses.sum()

```

Dual gap:

```

primal_loss = primal_objective(w_hat)
dual_loss = -dual_objective(alphaStar)[0]
duality_gap = primal_loss - dual_loss[0]
return w, b

```

CENTER DATA

```

def center_data(DTR, DVAL):
    mean = vcol(DTR.mean(axis = 1))
    DTR_c = DTR - mean
    DVAL_c = DVAL - mean

    return DTR_c, DVAL_c

```

Compute minDCF, actDCF for each value C to study effect of regularization.

```
C_values = np.logspace(-5, 0, 11)
```

```
def evaluate_linear_SVM_over_C(DTR, LTR, DVAL, LVAL, C_values, piT = 0.1):
    minDCF_values = []
    actDCF_values = []
    for C in C_values:
        w, b = train_dual_SVM_linear(DTR, LTR, C=C, K=1.0)
        SVAL = (vrow(w) @ DVAL + b).ravel() #s(x) = wTx + b
        minDCF = bayesRisk.compute_minDCF_binary_fast(
            SVAL, LVAL, piT, 1.0, 1.0
        )
        actDCF = bayesRisk.compute_actDCF_binary_fast(
            SVAL, LVAL, piT, 1.0, 1.0
        )
        minDCF_values.append(minDCF)
        actDCF_values.append(actDCF)

    return np.array(minDCF_values), np.array(actDCF_values)

# KERNEL POLINOMIALE
def polyKernel(degree, c):
    def polyKernelFunc(D1, D2):
        return (np.dot(D1.T, D2) + c) ** degree
    return polyKernelFunc

# RBF KERNEL
def rbfKernel(gamma):
    def rbfKernelFunc(D1, D2):
        # ||x-y||^2 = ||x||^2 + ||y||^2 - 2x^T y
        D1Norms = (D1**2).sum(0) # ||x_i||^2
        D2Norms = (D2**2).sum(0) # ||x_j||^2
        # ||x_i||^2 + ||x_j||^2
        Z = (vcol(D1Norms) + vrow(D2Norms) - 2*np.dot(D1.T, D2))
        K = np.exp(-gamma * Z)
        return K
    return rbfKernelFunc

# KERNEL FUNCTION
def train_dual_SVM_kernel(DTR, LTR, C, kernelFunc, eps=1.0):
    # eps: termine costante aggiunto al kernel per gestire il bias
    # 1. Convertiamo le label da {0,1} a {-1,+1}
    ZTR = LTR * 2.0 - 1.0
    # 2. Calcoliamo la kernel matrix:
    # K[i,j] = k(x_i, x_j) + eps
    K = kernelFunc(DTR, DTR) + eps
    # 3. Costruiamo la matrice H del duale:
    # H[i,j] = z_i z_j K[i,j]
    H = vcol(ZTR) * vrow(ZTR) * K
```

```

# 4. Definiamo il duale da minimizzare:
#   LD(alpha) = 1/2 alpha^T H alpha - alpha^T 1
def dual_objective(alpha):
    H_alpha = H @ vcol(alpha)
    loss = 0.5 * (vrow(alpha) @ H_alpha).ravel() - alpha.sum()
    grad = H_alpha.ravel() - np.ones(alpha.size)
    return loss, grad

# 5. Ottimizziamo alpha con vincoli:
#   0 <= alpha_i <= C
alpha0 = np.zeros(DTR.shape[1])
bounds = [(0, C) for _ in range(DTR.shape[1])]
alpha_star, _, _ = scipy.optimize.fmin_l_bfgs_b(dual_objective, alpha0,
bounds=bounds, factr=np.nan, pgtol=1e-5)

# 6. Definiamo una funzione per calcolare gli score sui dati futuri
def fScore(DTE):    Calcola gli score SVM sui dati DTE.
    score(x_t) = sum_i alpha_i z_i [k(x_i, x_t) + eps]
    K_test = kernelFunc(DTR, DTE) + eps
    scores = (vcol(alpha_star) * vcol(ZTR) * K_test).sum(axis=0)
    return scores
return fScore

def evaluate_poly_SVM_over_C(DTR, LTR, DVAL, LVAL, C_values, piT=0.1):
    minDCF_values = []
    actDCF_values = []
    kernelFunc = polyKernel(2, 1)
    for C in C_values:
        fScore = train_dual_SVM_kernel(DTR, LTR, C=C, kernelFunc=kernelFunc,
eps=0.0)
        SVAL = fScore(DVAL)
        minDCF = bayesRisk.compute_minDCF_binary_fast(
            SVAL, LVAL, piT, 1.0, 1.0
        )
        actDCF = bayesRisk.compute_actDCF_binary_fast(
            SVAL, LVAL, piT, 1.0, 1.0
        )
        minDCF_values.append(minDCF)
        actDCF_values.append(actDCF)

    return np.array(minDCF_values), np.array(actDCF_values)

```

Stessa identica funzione per RBF, basta solo sostituire la kernel function.

Project - question example 5

You are given the following functions (assume these functions are already implemented unless specified):

- `trainRBFKernelSVM(D, L, C, gamma)`: trains an SVM model with an RBF kernel with hyper-parameter `gamma` and returns an object containing the trained model information; `D` is the training data matrix, `L` is the corresponding label array, and `C` is the SVM cost-vs-margin trade-off coefficient
- `scoreRBFKernelSVM(svmModel, D)`: computes the classification scores for samples in the data matrix `D` for an SVM model `svmModel` (as returned by the function `trainRBFKernelSVM`) and returns an array of scores
- `evaluateScores(S, L)`: computes an evaluation metric (e.g. minimum DCF) over the array of scores `S` with associated array of labels `L`

Assume that you have at your disposal a training set, already divided in model training data (`DTR`, `LTR`) and validation data (`DVAL`, `LVAL`), and an evaluation set (`DTE`, `LTE`). `DTR`, `DVAL` and `DTE` are data matrices, with samples organized as column vectors, whereas `LTR`, `LVAL` and `LTE` are arrays containing the corresponding labels.

Write the Python code to train and apply an SVM classifier. In particular, the code should

- Train an SVM classifier, optimizing the value of the hyper-parameters with respect to the metric function `evaluateScores` using a single-fold cross-validation approach.
- Evaluate the selected SVM model on the evaluation data, using the provided metric.

Write an implementation of `scoreRBFKernelSVM(svmModel, D)`. Assume that `svmModel` is an object with the following fields:

`sv`: numpy 2-D array of support vectors, stored as column vectors
`alpha`: Lagrange multiplier values associated to each sv, as a 1-D numpy array
`labels`: 1-D numpy array of labels (+1 or -1) associated to the support vector
`gamma`: RBF kernel hyper-parameter γ

You can assume that you have at your disposal a function `RBFKernel(D1, D2, gamma)` that returns the matrix of kernel values $k(\mathbf{x}, \mathbf{y})$ for all pairs of samples $\mathbf{x}, \mathbf{y}$ of 2-D sample matrices `D1`, `D2` (i.e., if $K = \text{RBFKernel}(D1, D2, \text{gamma})$, then $K[i, j]$ is the kernel between arrays `D1[:, i]` and `D2[:, j]`).

import numpy as np

def scoreRBFKernelSVM(svmModel, D):

"""

Computes SVM scores for samples in D.

`svmModel.sv` : support vectors, shape (features, nSV)

`svmModel.alpha` : alpha values, shape (nSV,)

`svmModel.labels` : labels in {-1, +1}, shape (nSV,)

`svmModel.gamma` : RBF gamma

`D` : data matrix, shape (features, nSamples)

returns:

scores, shape (nSamples,)

"""

`K = RBFKernel(svmModel.sv, D, svmModel.gamma)`

`scores = (`

`svmModel.alpha.reshape(-1, 1)`

`* svmModel.labels.reshape(-1, 1)`

`* K`

`).sum(axis=0)`

possible hyper-parameters

```

C_values = np.logspace(-3, 2, 11)
gamma_values = [np.exp(-4), np.exp(-3), np.exp(-2), np.exp(-1)]

best_score = None
best_C = None
best_gamma = None
best_model = None

# model selection on validation set
for gamma in gamma_values:
    for C in C_values:

        svmModel = trainRBFKernelSVM(DTR, LTR, C, gamma)

        SVAL = scoreRBFKernelSVM(svmModel, DVAL)

        score = evaluateScores(SVAL, LVAL)

        if best_score is None or score < best_score:
            best_score = score
            best_C = C
            best_gamma = gamma
            best_model = svmModel

# retrain final model using training + validation data
DTRVAL = np.hstack([DTR, DVAL])
LTRVAL = np.hstack([LTR, LVAL])

finalModel = trainRBFKernelSVM(DTRVAL, LTRVAL, best_C, best_gamma)

# evaluation
STE = scoreRBFKernelSVM(finalModel, DTE)

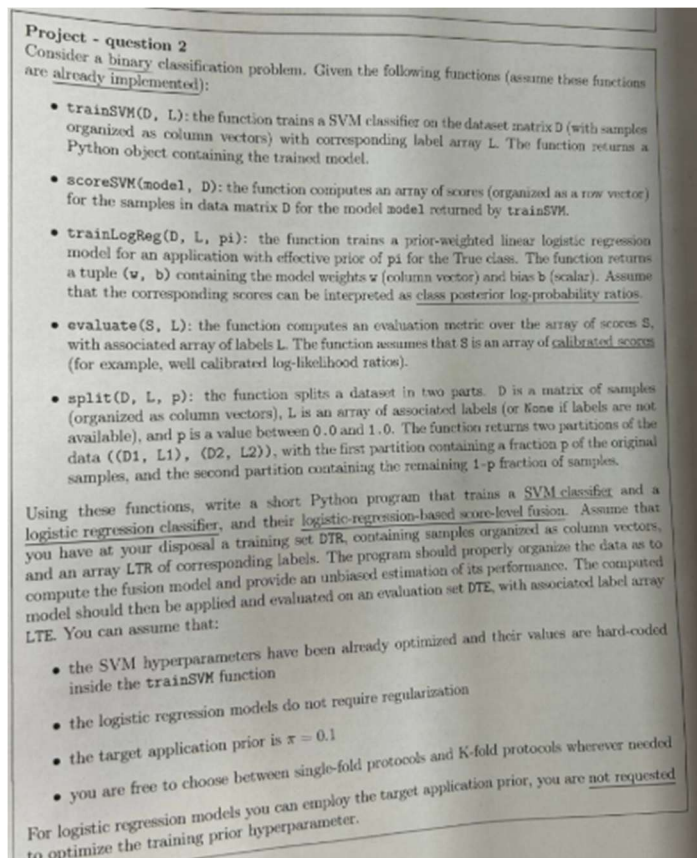
evalScore = evaluateScores(STE, LTE)

print("Best C:", best_C)
print("Best gamma:", best_gamma)

```

```
print("Validation score:", best_score)
print("Evaluation score:", evalScore)
```

return scores



I have a training dataset: DTR, LTR and a testing dataset: DTE, LTE.

First I split the dataset in training + validation:

(DTR_model, LTR_model), (DVAL, LVAL) = split(DTR, LTR, 0.7)

Then I train SVM and Logistic Regression on DTR, LTR:

svmModel = trainSVM(DTR_model, LTR_model)

w, b = trainLogReg(DTR_model, LTR_model, 0.1)

Then I compute the scores on validation set:

SVAL_svm = scoreSVM(svmModel, DVAL) → for svm

SVAL_lr = (w.T @ DVAL + b).ravel() → for lr

Finally I evaluate the models on the validation set: SVAL, LVAL

val_svm = evaluate(SVAL_svm, LVAL)

val_lr = evaluate(SVAL_lr, LVAL)

I retrain using all the training samples (DTR):

```

svmModel_final = trainSVM(DTR, LTR)
w_final, b_final = trainLogReg(DTR, LTR, 0.1)
And I test on the testing set: DTE:
STE_svm = scoreSVM(svmModel_final, DTE)
STE_lr = (w_final.T @ DTE + b_final).ravel()

eval_svm = evaluate(STE_svm, LTE)
eval_lr = evaluate(STE_lr, LTE)

```

In this case we use single cross validation:

TRAIN -> learn parameters

VALIDATION -> choose the model / estimate performance

TEST (evaluation) -> final unbiased performance

- Use part of the training data to train the model.
- Use the validation set to estimate performance / choose the model.
- Once you have finished model selection, train the final model using **all available training samples**.
- Then evaluate once on the evaluation set.

ESEMPI DOMANDE PROGETTO

Project – Question Example 1

1. Explain the relative performance of the MVG, Tied MVG and Naive Bayes models

Among the Gaussian classifiers, the best performance is achieved by the MVG and Naive Bayes models, while the Tied MVG model performs worse.

The obtained results are approximately:

Model	minDCF
MVG	≈ 0.26
Naive Bayes	≈ 0.26
Tied MVG	≈ 0.36

The MVG classifier models each class using its own covariance matrix: $x | c \sim \mathcal{N}(\mu_c, \Sigma_c)$

This allows the classifier to model different feature variances and correlations for the two classes, producing a more flexible quadratic decision boundary. The Tied MVG model instead assumes: $\Sigma_0 = \Sigma_1$ which forces both classes to share the same covariance matrix and produces a linear decision boundary. The worse performance of Tied MVG suggests that the two classes have different covariance structures and cannot be accurately modeled using a single shared covariance matrix. Naive Bayes achieves performance very close to MVG despite using diagonal covariance matrices. This indicates that feature

correlations are relatively weak. Indeed, the correlation matrices showed very small off-diagonal values, meaning that the conditional independence assumption is approximately valid for this dataset.

Therefore:

MVG performs well because it models class-specific covariance structures;

Tied MVG loses flexibility because of the shared covariance assumption;

Naive Bayes works surprisingly well because the features are only weakly correlated.

2. Explain the relative performance of linear and non-linear SVM

The obtained results are approximately:

Model	Best minDCF
Linear SVM	≈ 0.36
Polynomial SVM $d = 2$	≈ 0.25
Polynomial SVM $d = 4$	≈ 0.18
RBF SVM	≈ 0.18

Linear SVM achieves relatively poor performance compared to kernel SVMs. This indicates that the dataset is not linearly separable. The linear model learns a decision function of the form: $s(x) = w^T x + b$ which can only represent a linear separation boundary. However, the scatter plots of the last features reveal highly non-linear class structures, with multiple separated regions arranged approximately in a checkerboard/cross-like pattern. Polynomial kernels improve performance because they implicitly introduce non-linear feature interactions. In particular, the degree-2 kernel learns quadratic decision boundaries, while the degree-4 kernel can represent much more complex regions. The RBF kernel achieves the best overall performance because it can model highly local non-linear decision regions. The obtained results suggest that the project dataset contains strong non-linear relationships that cannot be captured by linear classifiers.

Therefore:

Linear SVM underfits the dataset;

Polynomial kernels improve performance by modeling non-linear interactions;

RBF kernels achieve the best results because they provide highly flexible non-linear decision boundaries.

Project – Question Example 2

Explain the relative performance of the different SVM kernels and analyze the effects of regularization

The different SVM kernels produce substantially different results:

Kernel	Best minDCF
Linear	≈ 0.36

Polynomial $d = 2$	≈ 0.25
Polynomial $d = 4$	≈ 0.18
RBF	≈ 0.18

The linear kernel performs worst because it can only model linear decision boundaries. The polynomial kernels improve the performance because they implicitly expand the feature space with higher-order interactions between features. The degree-4 polynomial kernel performs better than the degree-2 kernel because the dataset contains more complex non-linear structures that cannot be fully represented using only quadratic terms. The RBF kernel achieves the best performance because it can learn highly flexible local decision boundaries. The parameter C controls regularization:

small $C \rightarrow$ strong regularization,

large $C \rightarrow$ weak regularization.

For very small values of C :

minDCF actDCF

$\approx 1 \quad \approx 1$

the classifiers strongly underfit because the margin constraints dominate the optimization. As C increases, the models become more flexible and performance improves significantly. However, excessively large values of C may start degrading minDCF, especially for highly flexible kernels such as RBF or degree-4 polynomial kernels, suggesting mild overfitting. Therefore, intermediate values of C provide the best trade-off between underfitting and overfitting.

Project – Question Example 3

Explain the gap between minimum and actual DCF for SVM and Logistic Regression

For both Logistic Regression and SVM, we consistently observe:

$$\text{actDCF} > \text{minDCF}$$

This gap indicates imperfect score calibration.

The minimum DCF measures the best achievable performance after selecting the optimal threshold, whereas actual DCF evaluates the classifier using the theoretical Bayes threshold.

For Logistic Regression, strong regularization degrades calibration because the scores become compressed and lose their probabilistic interpretation.

In particular:

small regularization (λ small) preserves score quality;

large regularization (λ large) increases actDCF significantly.

For SVMs, the gap is typically larger because SVM scores are discriminative scores rather than calibrated log-likelihood ratios.

Linear SVM showed:

minDCF actDCF

$\approx 0.36 \approx 0.49$

whereas the best RBF SVM achieved:

minDCF actDCF

$\approx 0.18 \approx 0.32 - 0.38$

Therefore, the more flexible non-linear kernels improve discrimination capability, but calibration issues still remain.

The calibration gap is generally reduced when the model better captures the true data structure.

Regarding regularization: excessive regularization causes underfitting and compressed scores; insufficient regularization may increase overfitting; intermediate regularization values provide the best balance between discrimination and calibration. Overall, both Logistic Regression and SVM exhibit calibration errors, but the non-linear SVM models achieve much stronger discriminative performance because they better model the highly non-linear structure of the dataset.